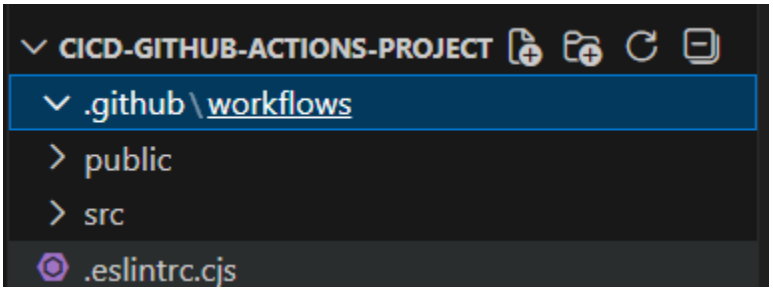
GITHUB ACTIONS END TO END DEPLOYMENT

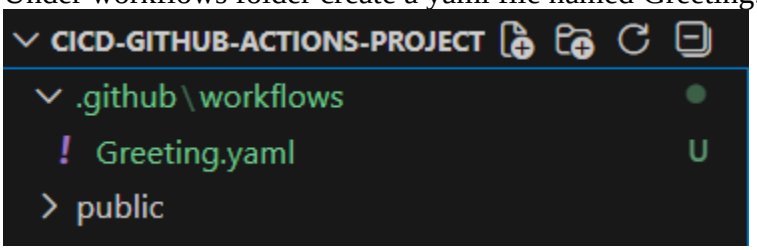
- Git repo: https://github.com/akshu20791/cicd-github-actions-project.git
- clone the Github repo in your local system
- Create a new Github repo and Push the code

1. **First we have to print "Hello word" in Git action**

- Create a folder → .github/workflows (in cloned repository)



- Under workflows folder create a yaml file named Greeting.yaml



Go to Extensions in Vscode and Install Github Extension

Now write in yaml file to run Hello World

Code:-

```
name: Greeting Workflow
on: push
jobs:
  greet:
        runs-on: ubuntu-latest
        steps:
          - name: Greet User
            run: echo "Hello, World!"
```

greet is the **job name** (GitHub Actions uses the word job

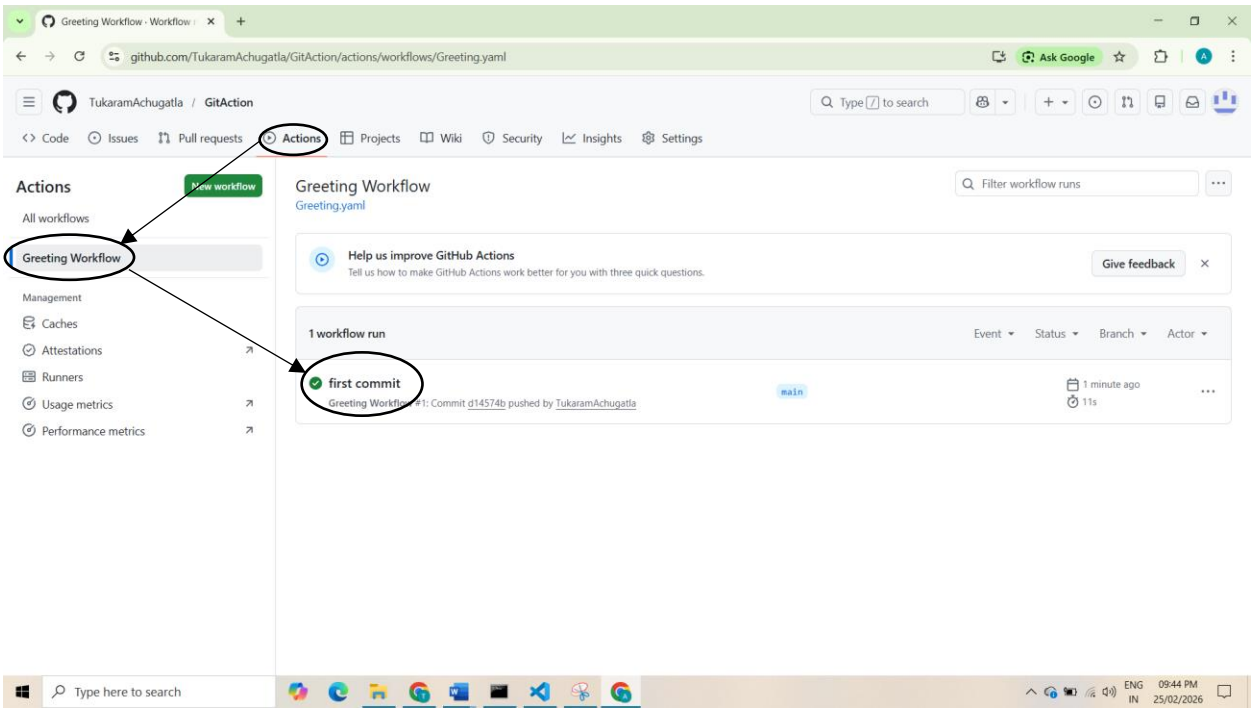
what is **runs-on: ubuntu-latest**?

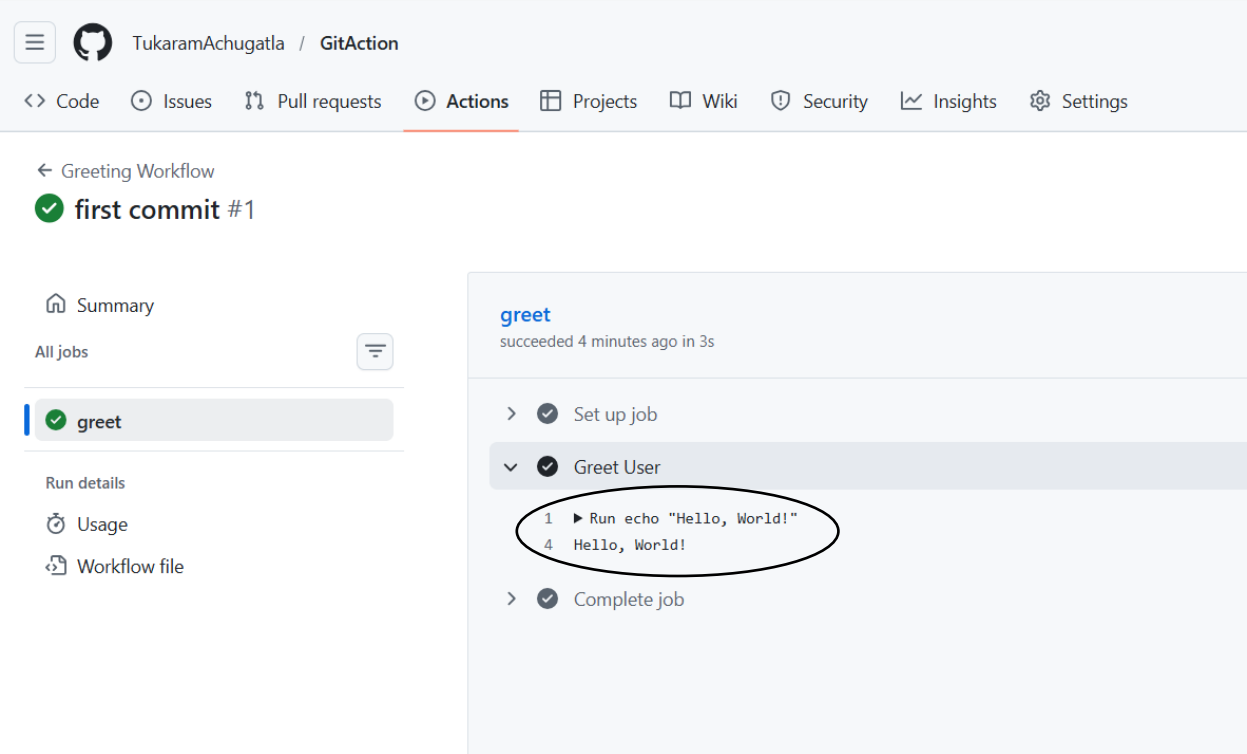
This tells GitHub **which machine (runner) to use** to execute the job.

- `ubuntu-latest` → Runs on the latest Ubuntu Linux virtual machine.
- It is a **GitHub-hosted runner**.
- GitHub automatically provides this temporary server.

Now Add the code to Github repository and Push the code

Open GitHub→ Click on Action →Gretting Workflow →first commit/greet → Greet User





It will Trigger Automatically without any Button if you need button do the following steps(Optional)

Step 1: Information about trigger
Triggers are defined in : https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows

Step 2: Key to add Button
workflow_dispatch: can be used to manually get a button to run the github action workflow.

Ex:-

```
name: Greeting Workflow
on: [push,workflow_dispatch]
jobs:
```

2. **We need to write the workflow where in the github runners first we will clone the github repo**

- Create a yaml file in workflows
- There are 2 method to clone the git repo in git action

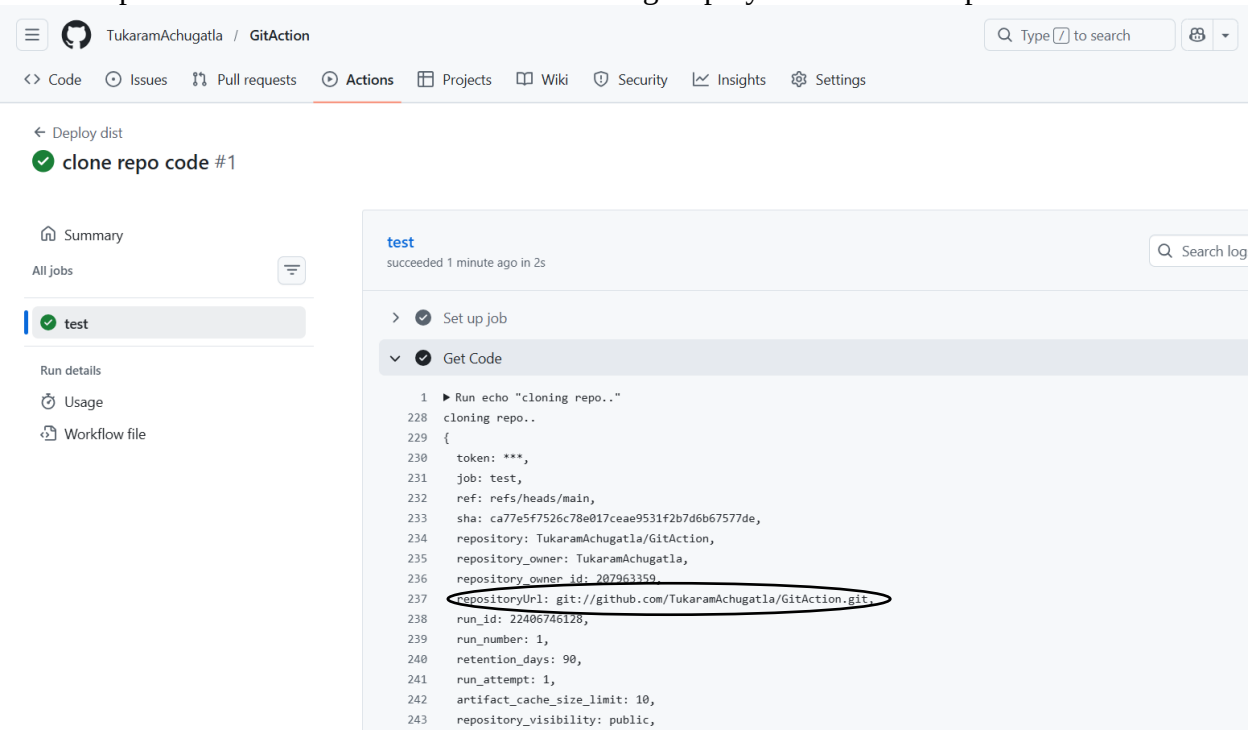
a. Difficult Method

```
name: Deploy dist
on: push
jobs:
   test:
      runs-on: ubuntu-latest
      steps:
         - name: Get Code
           run: |
             echo "cloning repo.."
             echo "${{toJson(github)}}"
             git clone https://github.com/${{github.repository}}.git
```

${{ }}= Expression Syntax
toJson(github)= converts the entire github object into JSON format
github.repository= your owner/repository-name

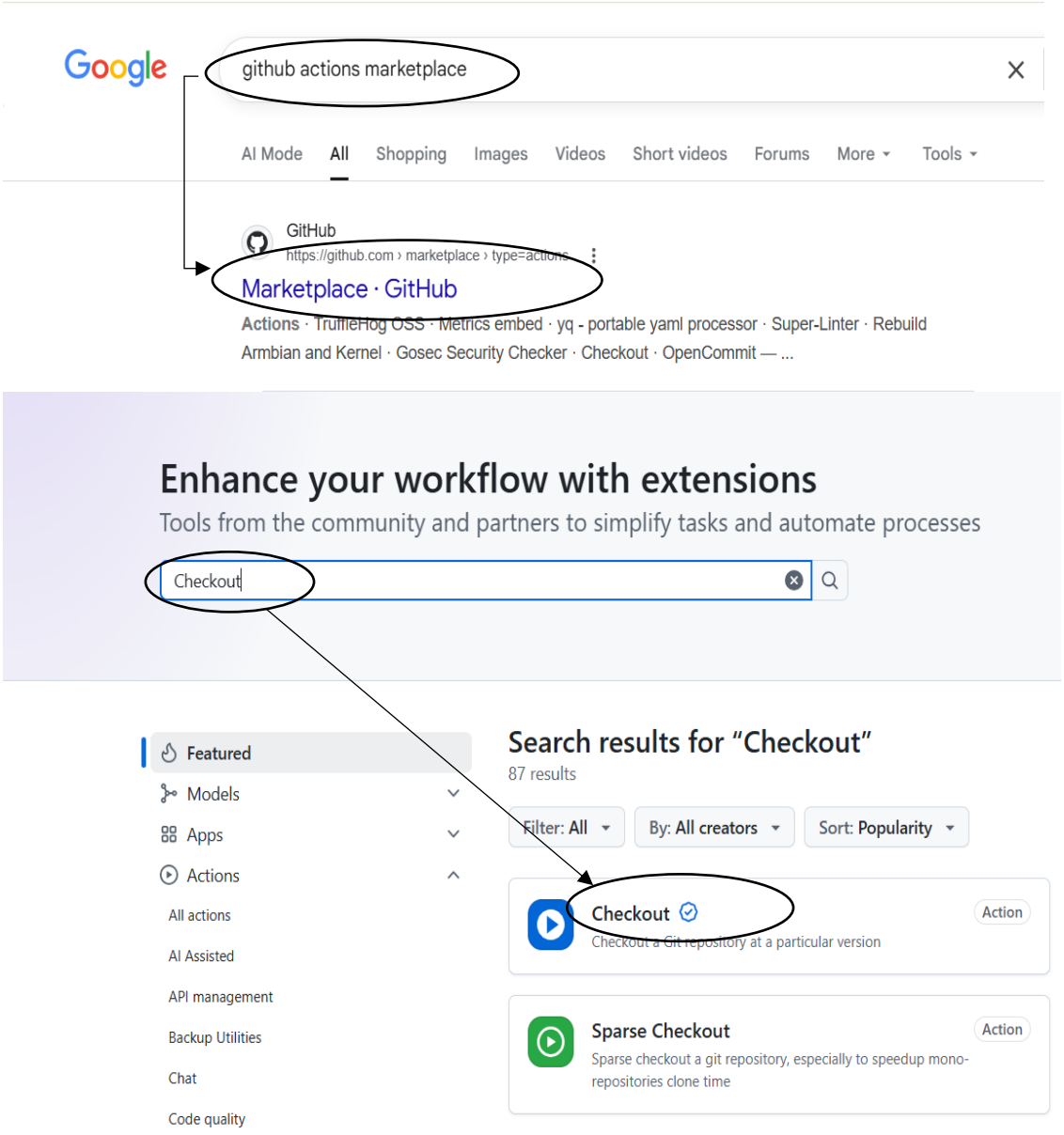
Now Add the code to Github repository and Push the code

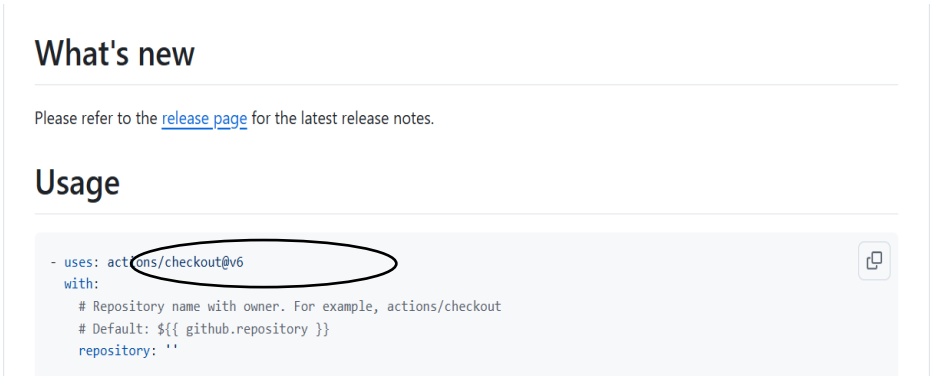
Open GitHub→ Click on Action →Gretting Deploy dist → clone repo code → test



b. Easy Method

Search for github action marketplace in google→click on Marketplace → search checkout → click on Checkout → Scroll Down see Usage → Copy - user: actions/checkout@v6





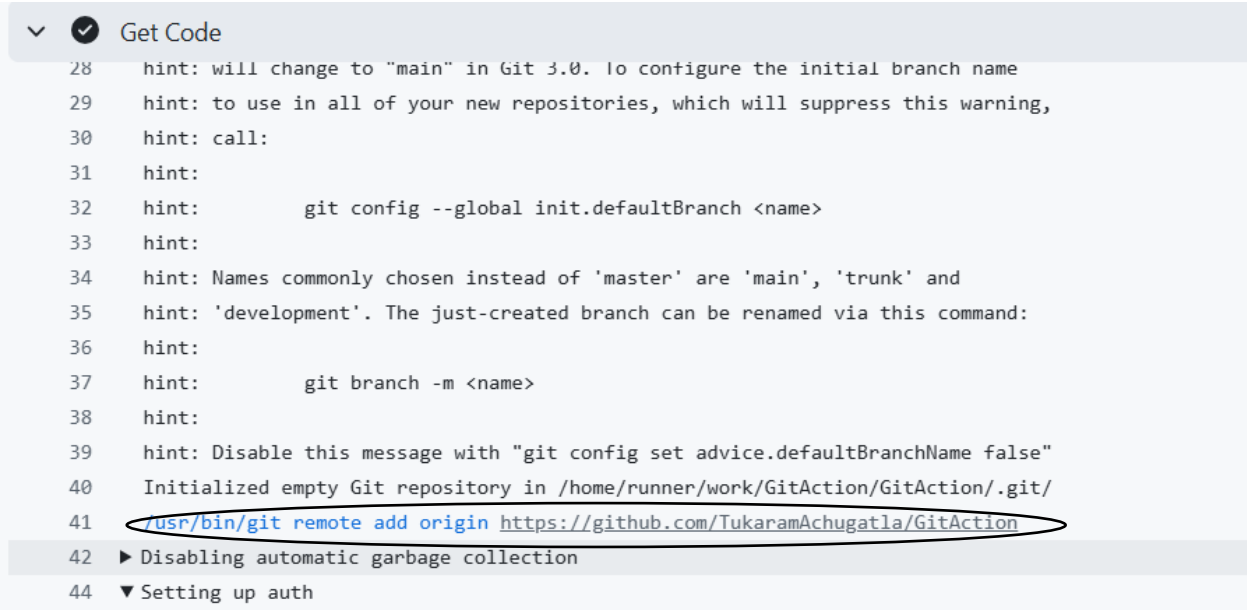
Code :

```
name: Deploy dist
on: [push,workflow_dispatch]
jobs:
test:
  runs-on: ubuntu-latest
  steps:
    - name: Get Code
      uses: actions/checkout@v6
```

What is **uses: actions/checkout@v6** ?

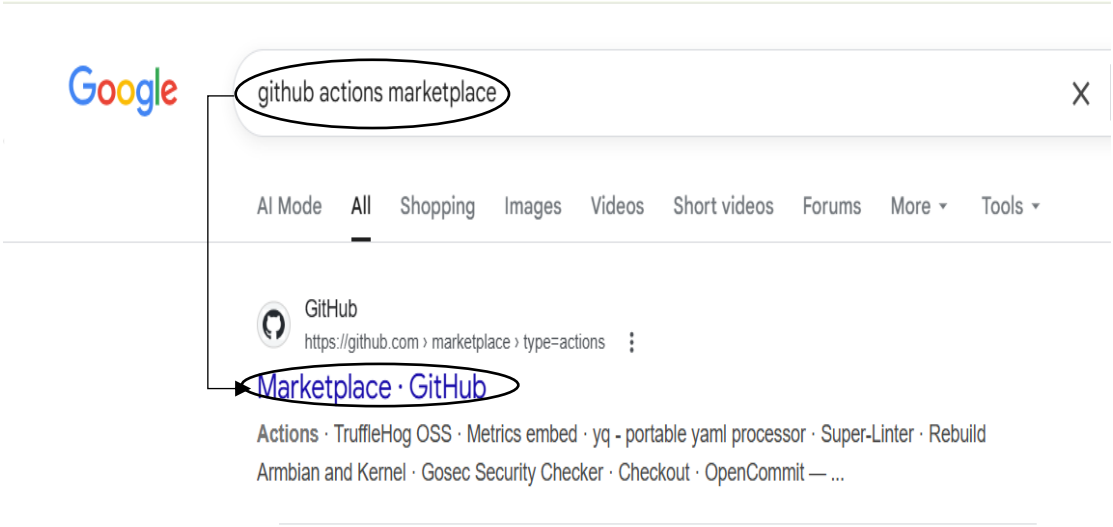
Use a pre-built GitHub Action called checkout (version 6)
This action downloads (clones) your repository code into the GitHub runner

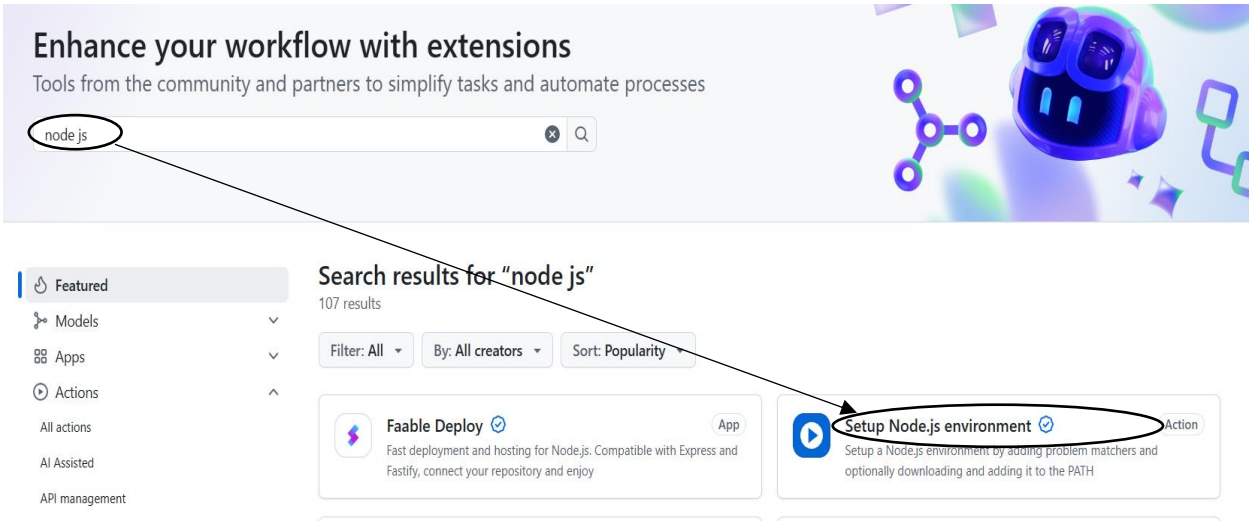
- As soon as we push the code to GitHub, the workflow will automatically execute.

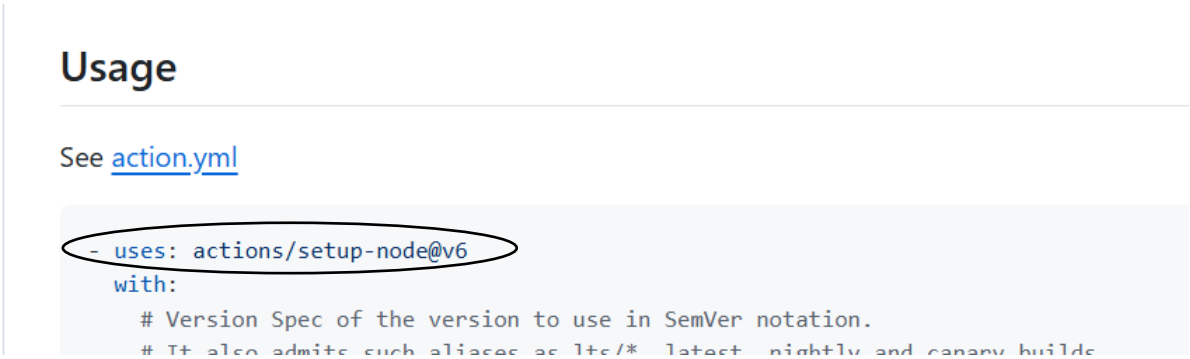


3. **Now we need to install npm in the github runner**

   Search for github action marketplace in google→click on Marketplace → search node js → click on Setup Node.js environment → Scroll Down see Usage → Copy - user: actions/setup-node@v6







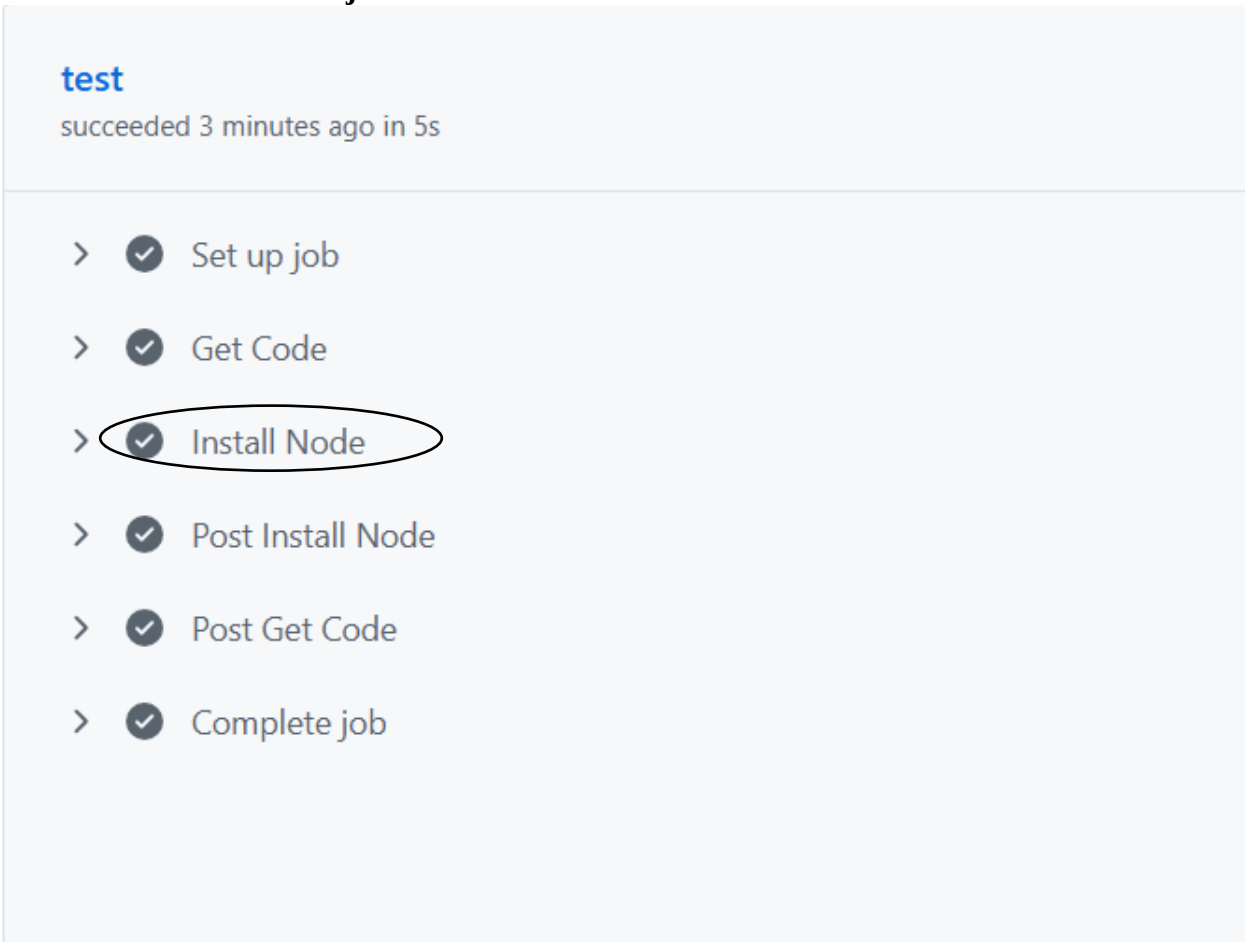
Code:-

```
name: Deploy dist
on: [push,workflow_dispatch]
jobs:
   test:
   runs-on: ubuntu-latest
   steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
            node-version: '20'
```

What is **uses: actions/setup-node@v6**?

It is an official GitHub Action

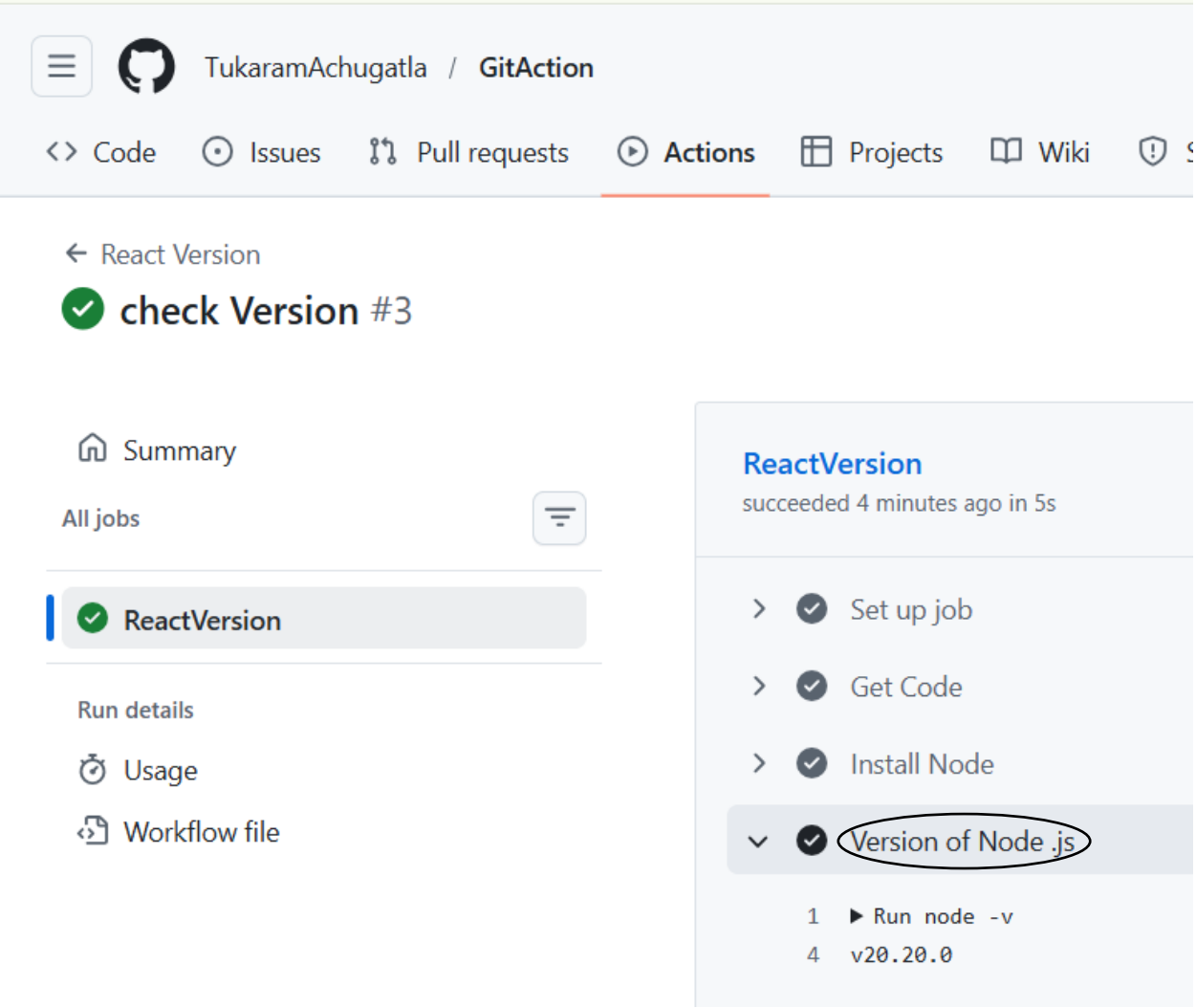
It installs **Node.js** in the GitHub runner



4. **Now let's print version of Node.js**

Code :-

```
name: React Version
on: [push,workflow_dispatch]
jobs:
   ReactVersion:
         runs-on: ubuntu-latest
         steps:
           - name: Get Code
             uses: actions/checkout@v6
           - name: Install Node
             uses: actions/setup-node@v6
             with:
                node-version: '20'
           - name: Version of Node .js
                run: node -v
```



5. **Now let's install Dependence in React Project**

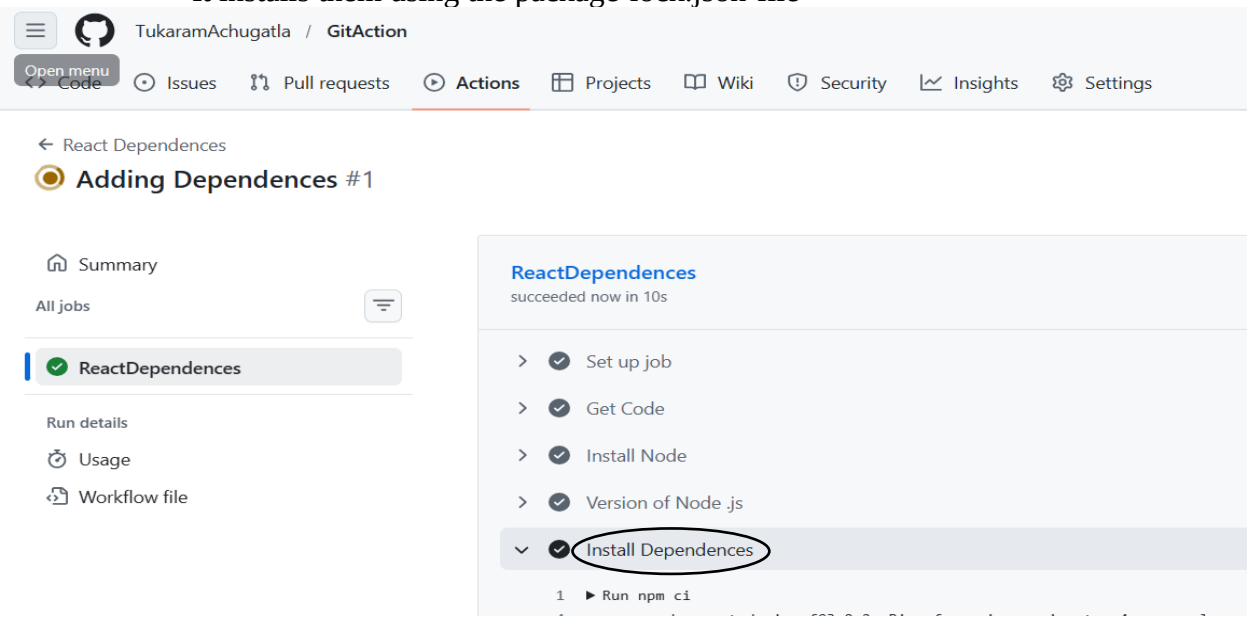
npm install= Installs dependencies from **package.json**
npm ci= Installs exactly what is in **package-lock.json**

Code:-

```
name: React Version
on: [push,workflow_dispatch]
jobs:
   ReactVersion:
        runs-on: ubuntu-latest
        steps:
          - name: Get Code
            uses: actions/checkout@v6
          - name: Install Node
            uses: actions/setup-node@v6
            with:
                node-version: '20'
         -   name: Version of Node .js
                run: node -v
          - name: Install Dependences
                run: npm ci
```

What does **run: npm ci** mean?

It runs the command npm ci inside the GitHub runner
It installs all project dependencies
It installs them using the package-lock.json file



Fail Test Case and Skip Workflow

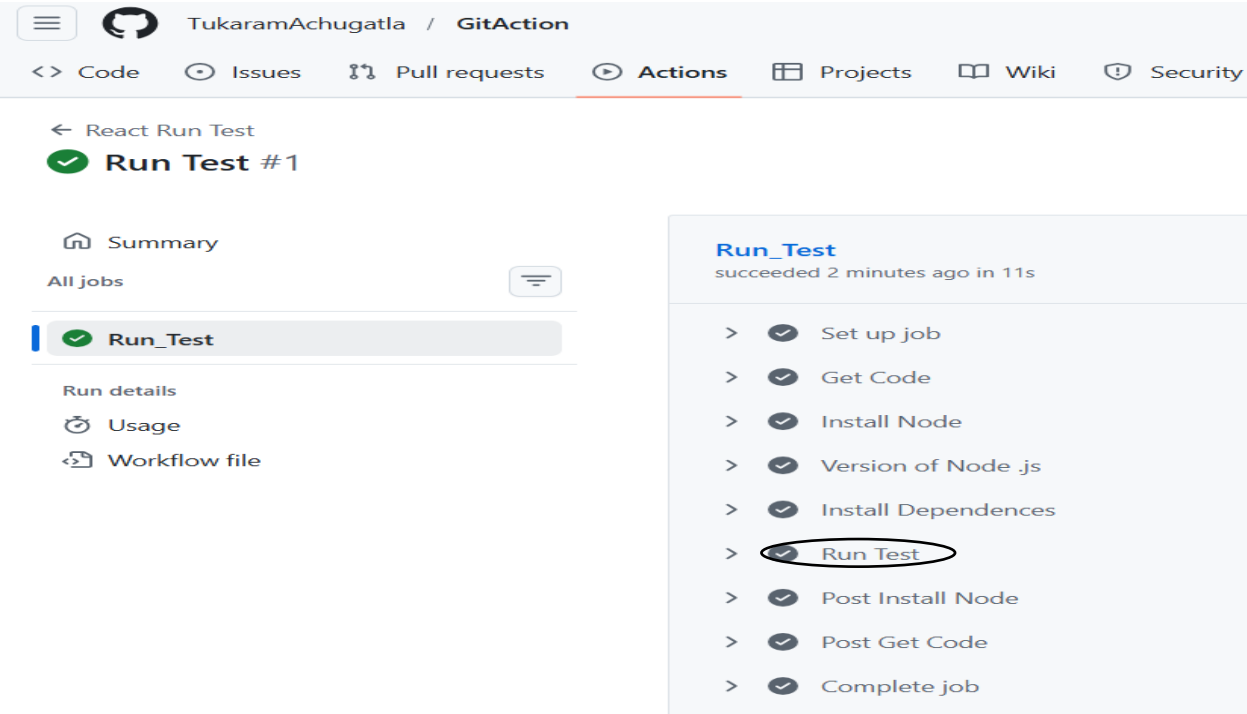
6. **Now let's Run Test**

Code:-

```
name: React Run Test
on: [push,workflow_dispatch]
jobs:
    Run_Test:
        runs-on: ubuntu-latest
        steps:
          - name: Get Code
            uses: actions/checkout@v6
          -  name: Install Node
            uses: actions/setup-node@v6
            with:
                node-version: '20'
          - name: Version of Node .js
                run: node -v
          - name: Install Dependences
                run: npm ci
          - name: Run Test
                run: npm test
```

What does **run: npm test** mean?

It runs the test script of your project
It checks whether your application is working correctly
It runs inside the GitHub runner

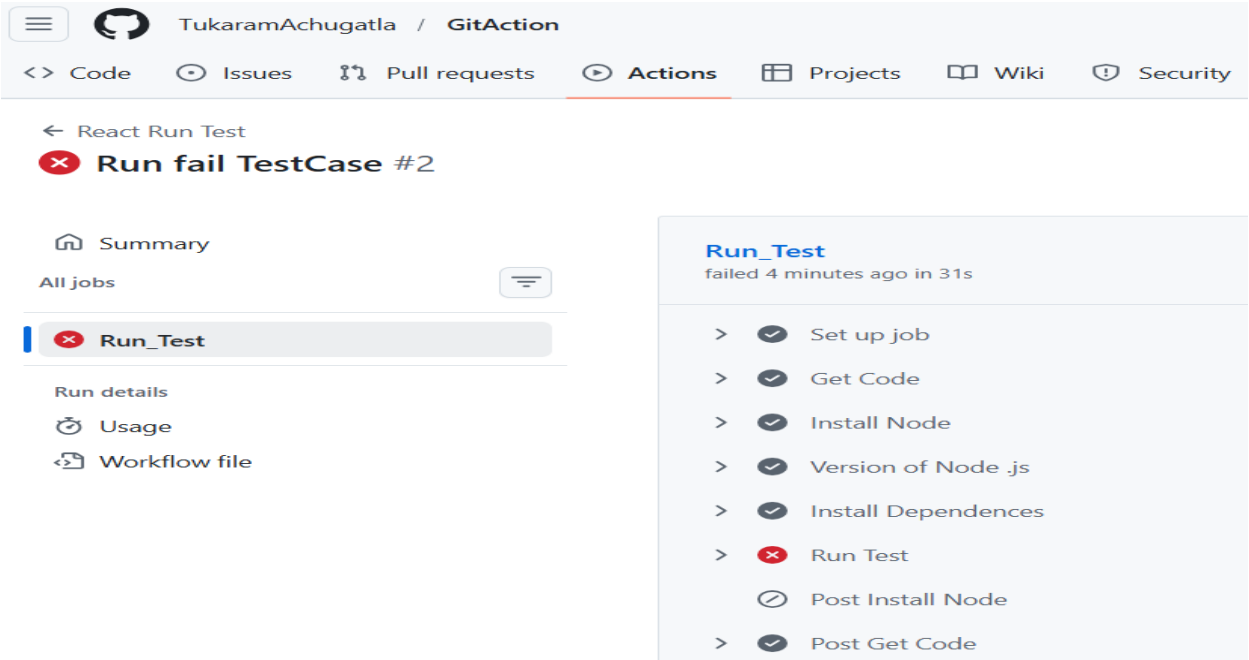


7. **Now let's do fail test case and it will show in red colour**

   Open src → open App.text.jsx→ on line number 21 add .not before .to
   (expect(h2Element).not.toBeInTheDocument();)
   Add the code to github and push the code

   Code :-

```
name: React Run Test
on: [push,workflow_dispatch]
jobs:
  Run_Test:
     runs-on: ubuntu-latest
     steps:
        - name: Get Code
          uses: actions/checkout@v6
        - name: Install Node
          uses: actions/setup-node@v6
          with:
            node-version: '20'
        - name: Version of Node .js
            run: node -v
        - name: Install Dependences
              run: npm ci
        - name: Run Test
              run: npm test
```



- **Note:-**

  If I add this comment in the commit message, Git should not trigger the workflow. So we can use: git commit -m "added commit [skip ci]".

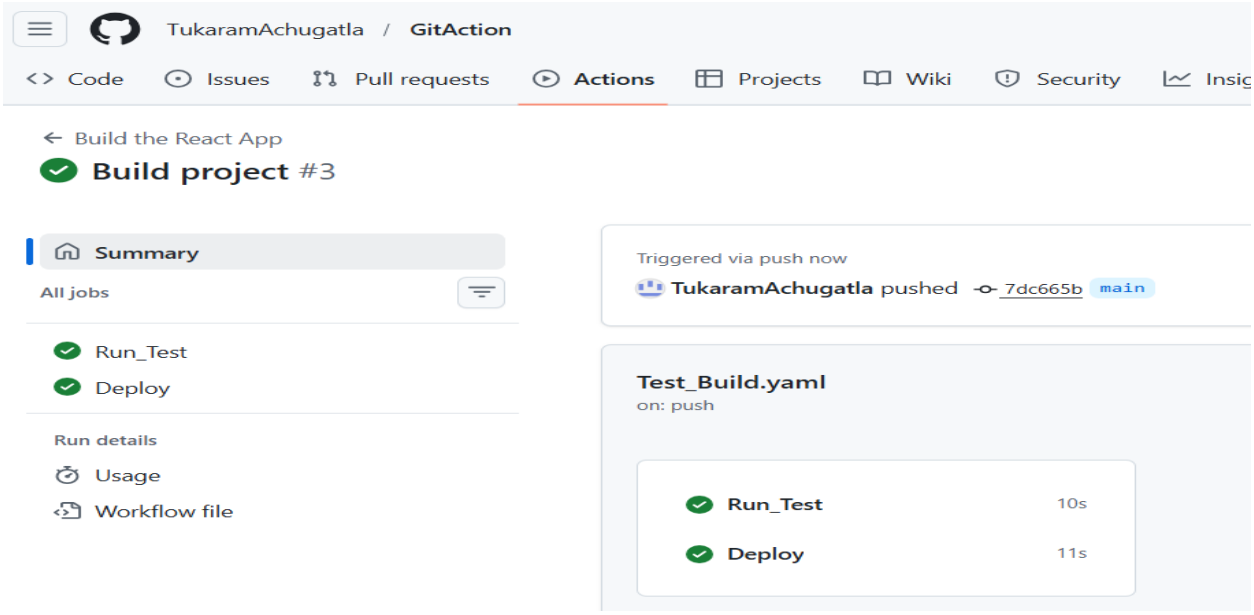
Paraller and Sequential Jobs

8. **Now let's run the tests, build, and deploy the project.**
   Code:-

```
name: Build the React App
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Install Dependences
        run: npm ci
      - name: Build Project
        run: npm run build
      - name: Deploy
        run: echo "Deploying to production server..."
```

   Run_Test **and** Deploy **are stages** in your workflow

   Run_Test stage/job → installs, then runs npm test
   Deploy stage/job → installs, builds, then deploys



Run_Test

succeeded 1 minute ago in 10s

- Set up job
- Get Code
- Install Node
- Version of Node .js
- Install Dependences
- Run Test
- Post Install Node
- Post Get Code
- Complete job

Deploy

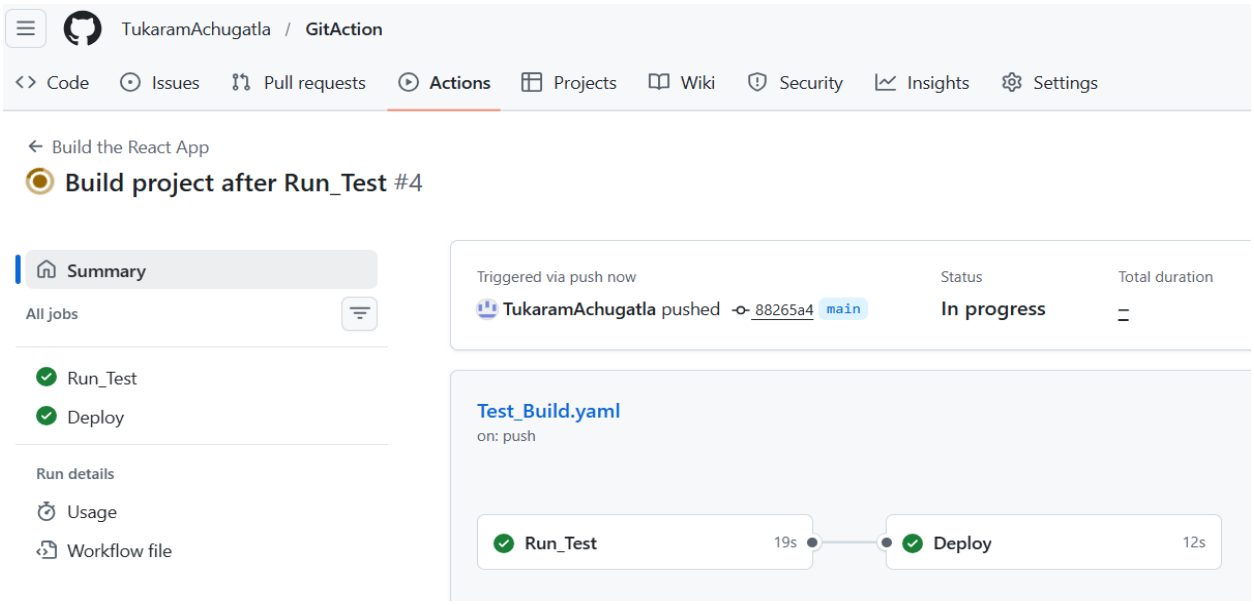
succeeded 1 minute ago in 11s

- Set up job
- Get Code
- Install Node
- Install Dependences
- Build Project
- Deploy
- Post Install Node
- Post Get Code
- Complete job

*****Note:-** It will run the Run_Test and Deploy in paraller. If we want to run Deploy after Run_Test then we need to add (needs: Run_Test) in the Deploy Job

Code :-

```
name: Build the React App
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Deploy:
    needs: Run_Test
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Install Dependences
        run: npm ci
      - name: Build Project
        run: npm run build
      - name: Deploy
        run: echo "Deploying to production server..."
```

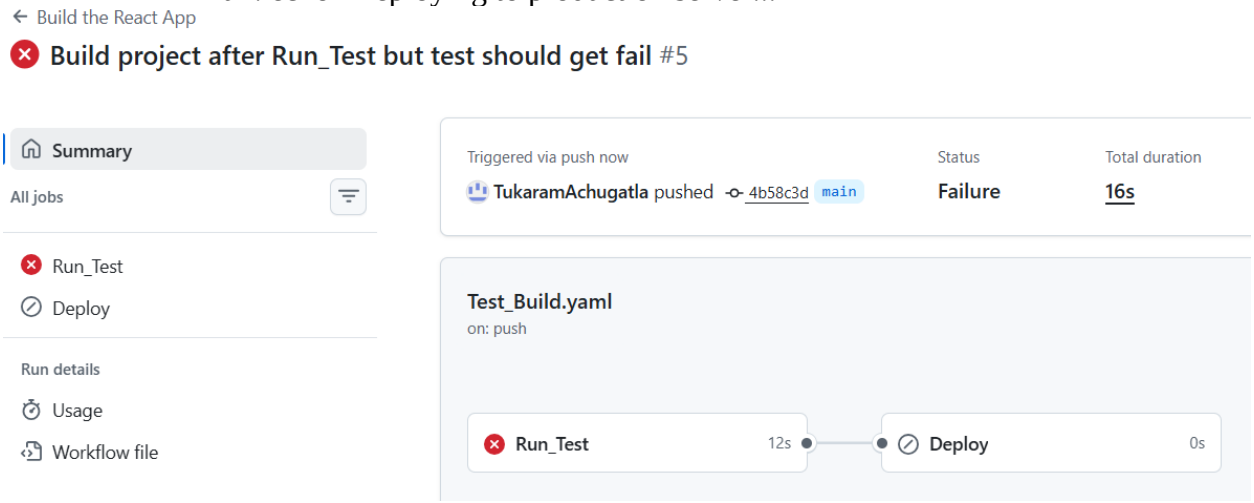


Now If the test fails, then it should not deploy

- Open src → open App.text.jsx→ on line number 21 add .not before .to
  (expect(h2Element).not.toBeInTheDocument();)
- Add the code to github and push the code
  Code:-

```
name: Build the React App
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Deploy:
    needs: Run_Test
    runs-on: ubuntu-latest
    steps:
```

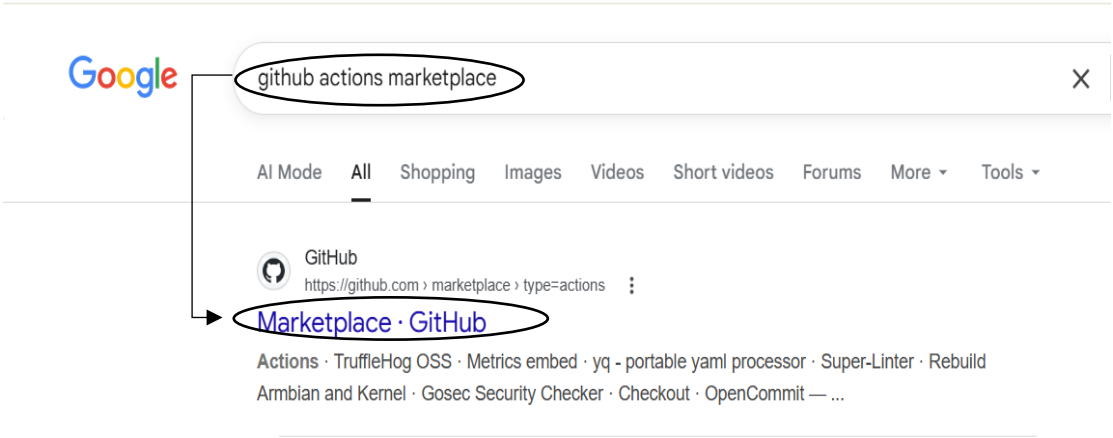
```
- name: Get Code
  uses: actions/checkout@v6
- name: Install Node
  uses: actions/setup-node@v6
  with:
    node-version: '20'
- name: Install Dependences
  run: npm ci
- name: Build Project
  run: npm run build
- name: Deploy
  run: echo "Deploying to production server..."
```

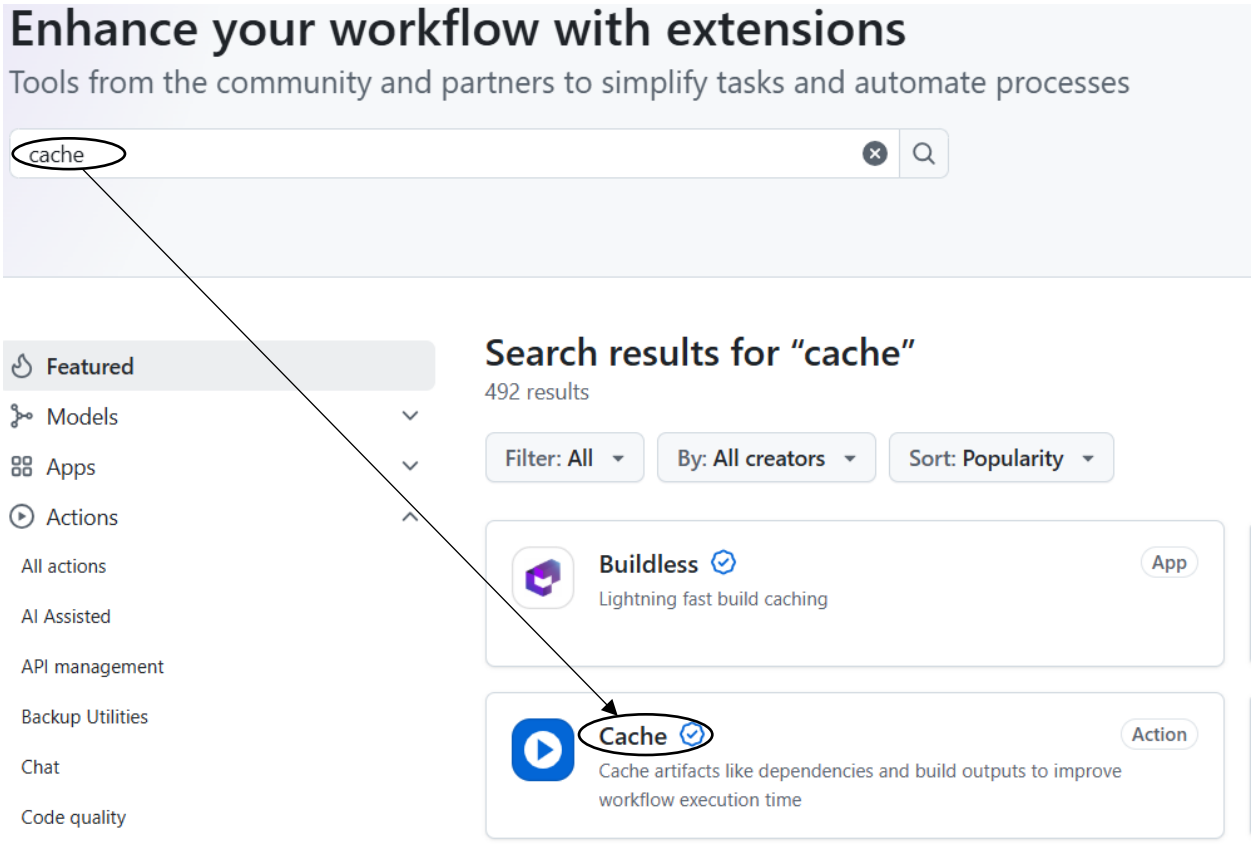


Cache Dependency

In Run_Test and Deploy, we are installing the same dependencies, which takes 7 seconds. To reduce that time, we can use dependency caching.

Let's got to Marketplace → search cache → scroll down for Node -npm → copy – user: actions/cache@v5 → copy key → copy path




Enhance your workflow with extensions
Tools from the community and partners to simplify tasks and automate processes
cache
Featured
Models
Apps
Actions
All actions
AI Assisted
API management
Backup Utilities
Chat
Code quality
Search results for "cache"
492 results
Filter: All
By: All creators
Sort: Popularity
Buildless
Lightning fast build caching
App
Cache
Cache artifacts like dependencies and build outputs to improve workflow execution time
Action

Implementation Examples

Every programming language and framework has its

See Examples for a list of `actions/cache` implementa

- Bun
- C# - NuGet
- Clojure - Lein Deps
- D - DUB
- Deno
- Elixir - Mix
- Go - Modules
- Haskell - Cabal
- Haskell - Stack
- Java - Gradle
- Java - Maven
- Node - npm
- Node - Lerna
- Node - Yarn
- OCaml/Reason - esy

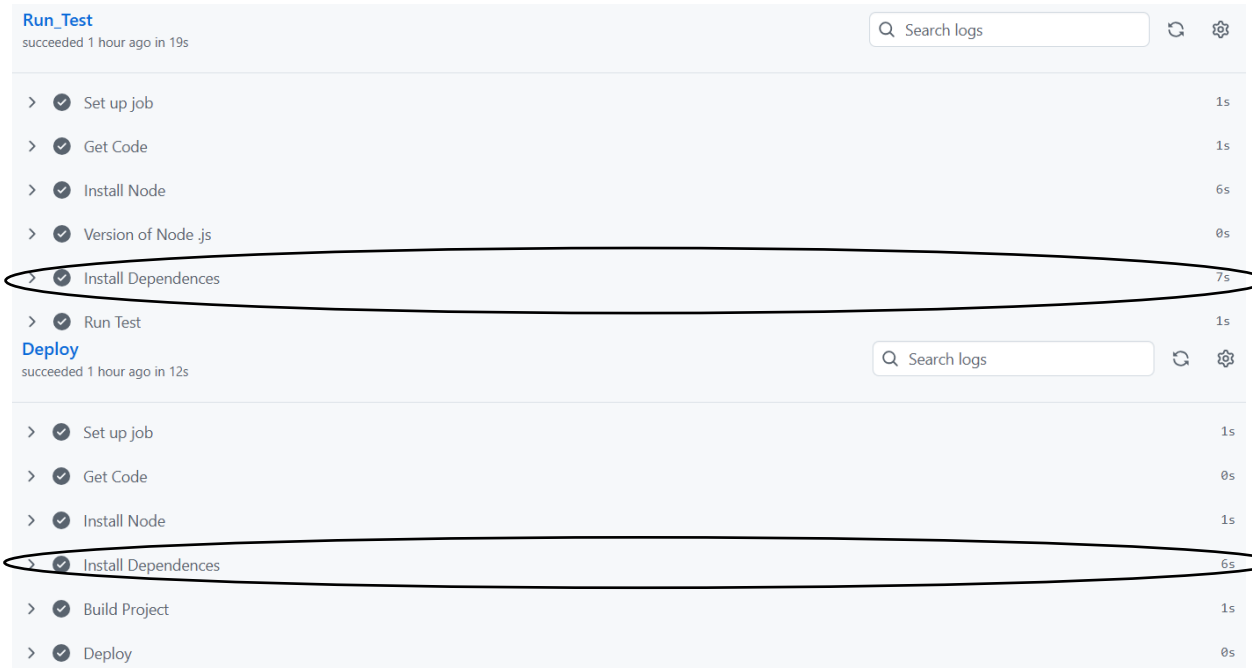
Code:-

```
name: Cache Depencences
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Cache Node Modules
        uses: actions/cache@v5
        with:
          path: ~/.npm
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Deploy:
    needs: Run_Test
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
```

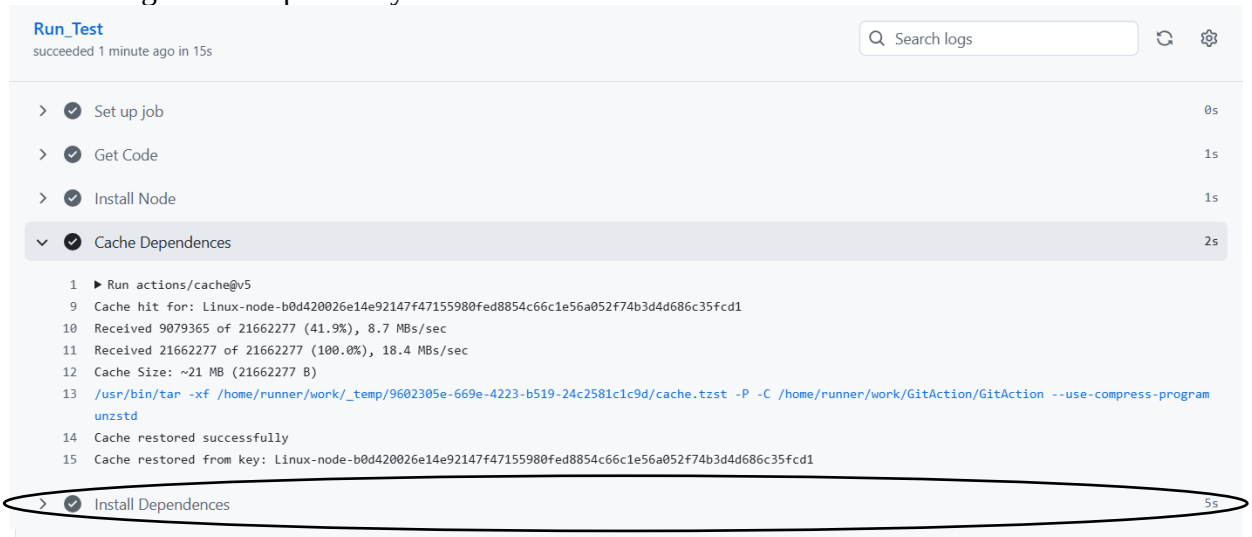
It will Generate a unique key based on the OS and the hash of the package-lock.json file

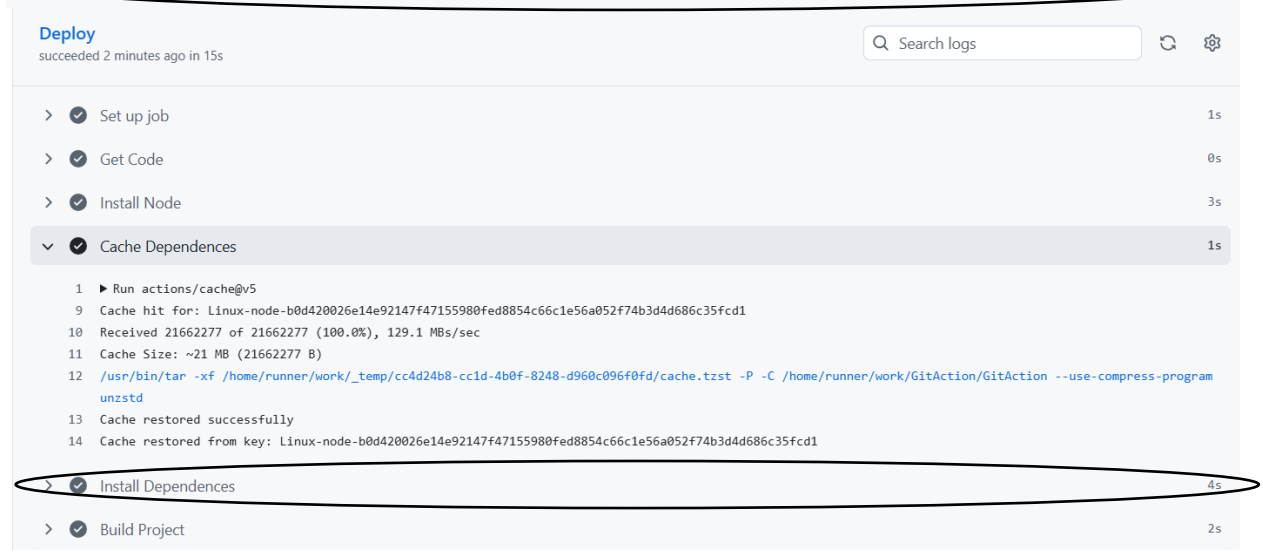
```
    uses: actions/setup-node@v6
    with:
      node-version: '20'
  - name: Cache Node Modules
    uses: actions/cache@v5
    with:
      path: ~/.npm
      key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
  - name: Install Dependences
    run: npm ci
  - name: Build Project
    run: npm run build
  - name: Deploy
    run: echo "Deploying to production server..."
```

Before::



After adding cache Dependency::





Filter On Event Push

Now, the workflow should trigger only when code is pushed to the main branch or feature.
Explain:-

```
on:
  push:
    branches:
      - main
      - feature/**
    paths-ignore:
      - README.md
      - '.github/workflows/**'
```

Example: -
This means the workflow will trigger when you push to
branches:
- main:- main branch only
- feature/** :- Any branch starting with feature(like → feature/login, feature/payment, etc)

The workflow will NOT run if only these files are changed
paths-ignore:
- README.md
- '.github/workflows/**'

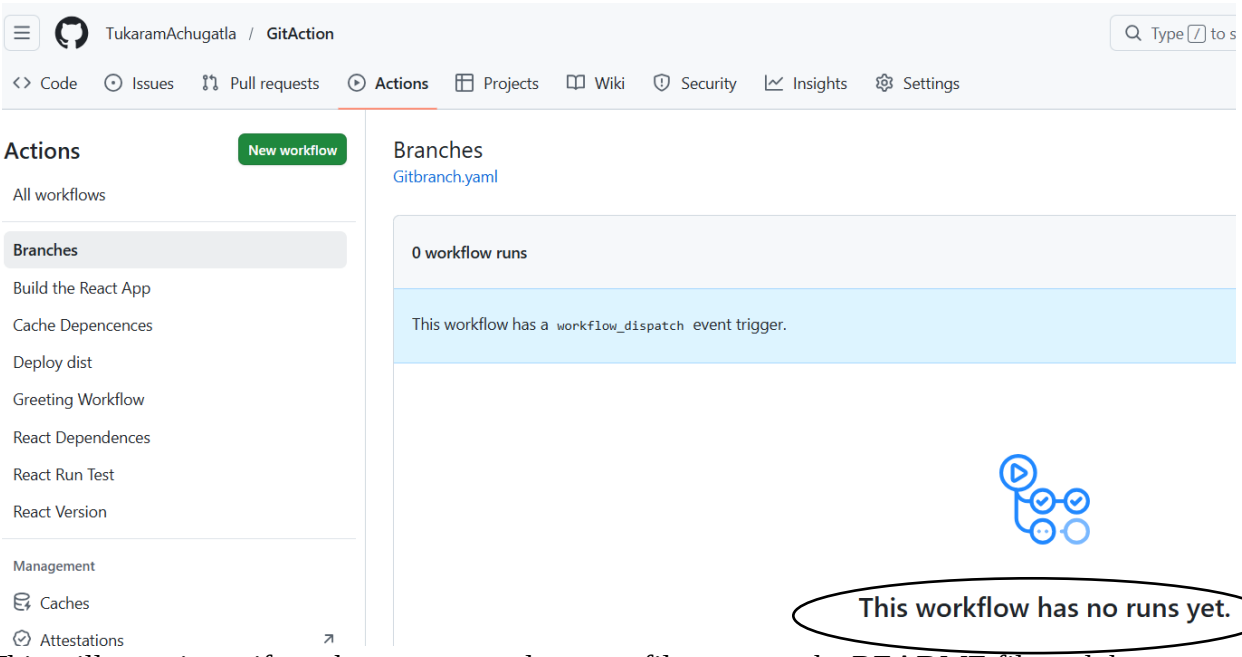
- Only README.md → ❌ Workflow will NOT trigger
- Only workflow file change → ❌ Workflow will NOT trigger
- Code file change (like `App.jsx`) → ✅ Workflow will trigger

So, Lets check first without making any changes in App.js file will be it trigger or Not

Code :-

```
name: Branches
# on: [push,workflow_dispatch]
on:
  push:
    branches:
      - main
      - feature/**
    paths-ignore:
      - README.md
      - '.github/workflows/**'
  workflow_dispatch:
jobs:
    Run_Test:
       runs-on: ubuntu-latest
       steps:
          - name: Get Code
            uses: actions/checkout@v6
          - name: Install Node
            uses: actions/setup-node@v6
            with:
              node-version: '20'
          - name: Cache Dependences
            uses: actions/cache@v5
            with:
              path: ~/.npm
              key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json')
```

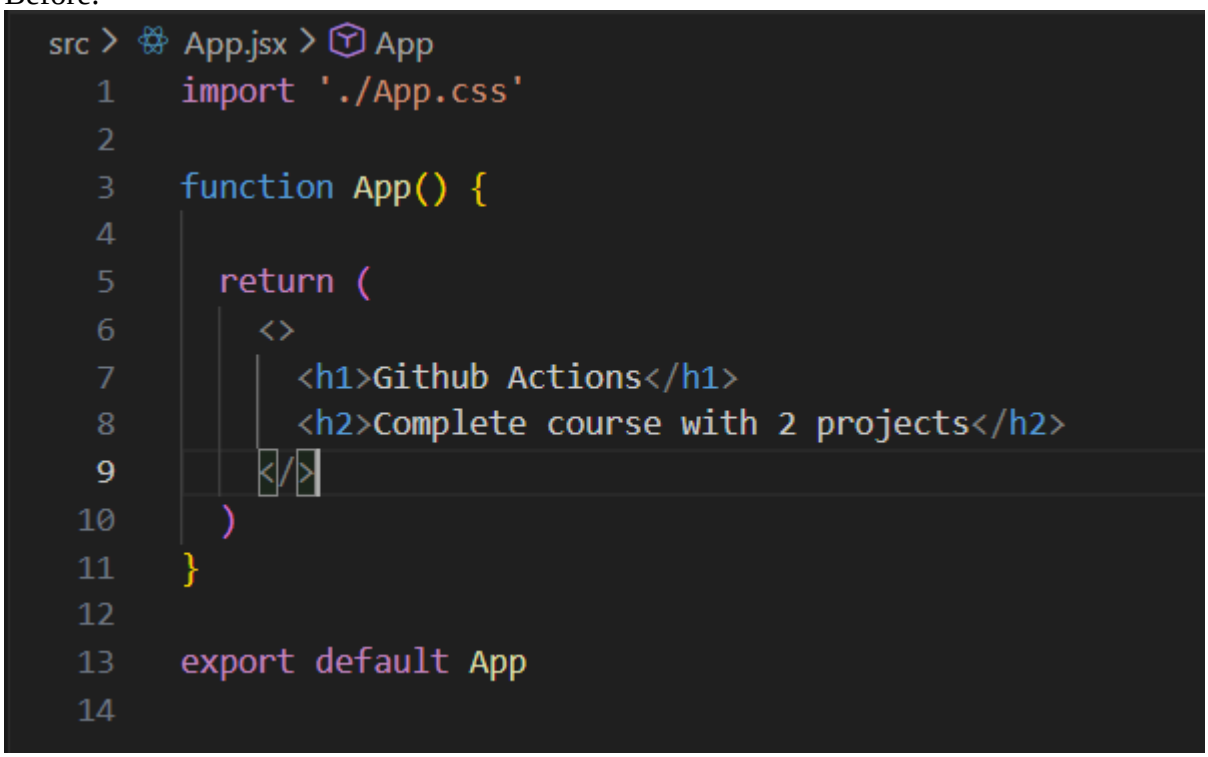
```
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Deploy:
    needs: Run_Test
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Cache Dependences
        uses: actions/cache@v5
        with:
          path: ~/.npm
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Install Dependences
        run: npm ci
      - name: Build Project
        run: npm run build
      - name: Deploy
        run: echo "Deploying to production server..."
```



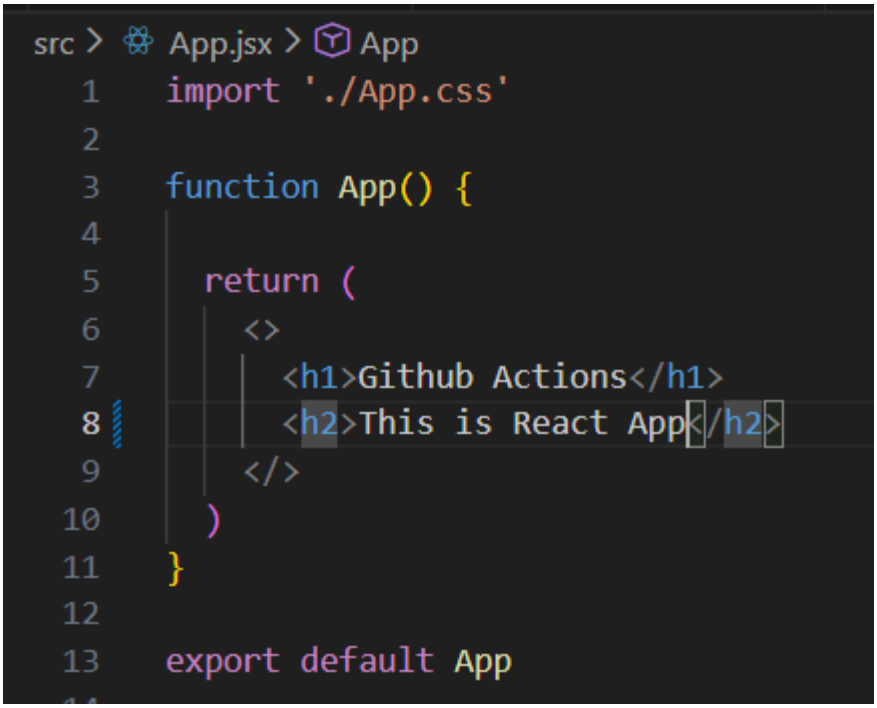
This will not trigger if no changes are made to any files, except the README file and the .github/workflows directory.

So, Let’s make changes in App.jsx file to h2.tag (“This is React App”)

Before:-

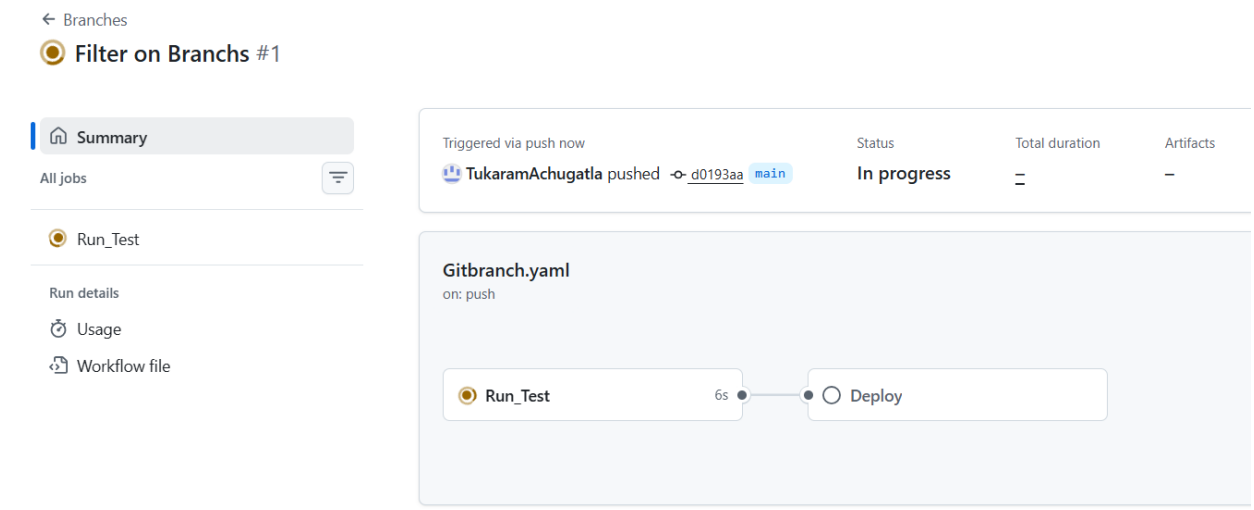


After:-



Now add App.jsx Code to Github And Push the code

Now the git is Trigger



Upload and Download artifact

Explain

What is an Artifact?

An **artifact** is a file or folder generated during a workflow that we want to Save temporarily, Share between jobs, Download later.

Examples:

1. Test reports
2. JAR/WAR files

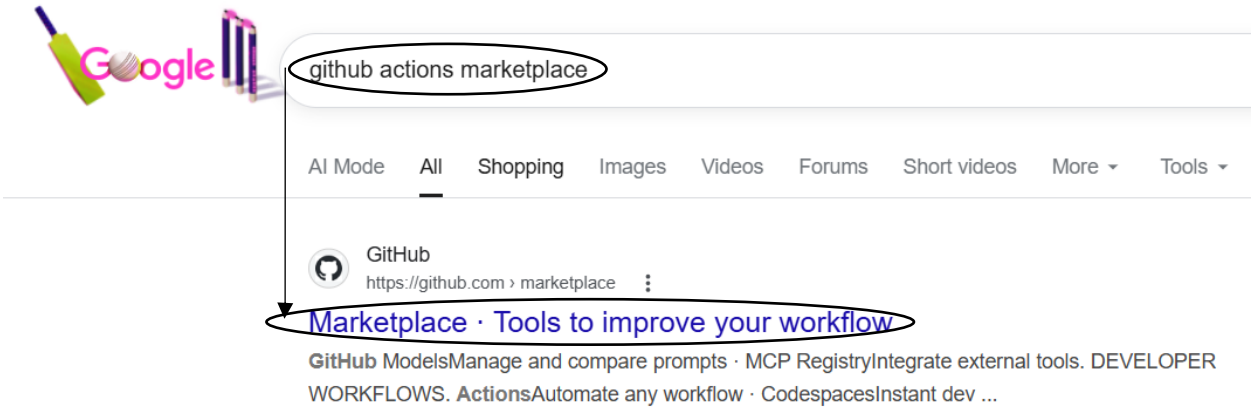
Why Do We Use Artifacts?

To transfer build output or important files from one job to another job in a workflow.

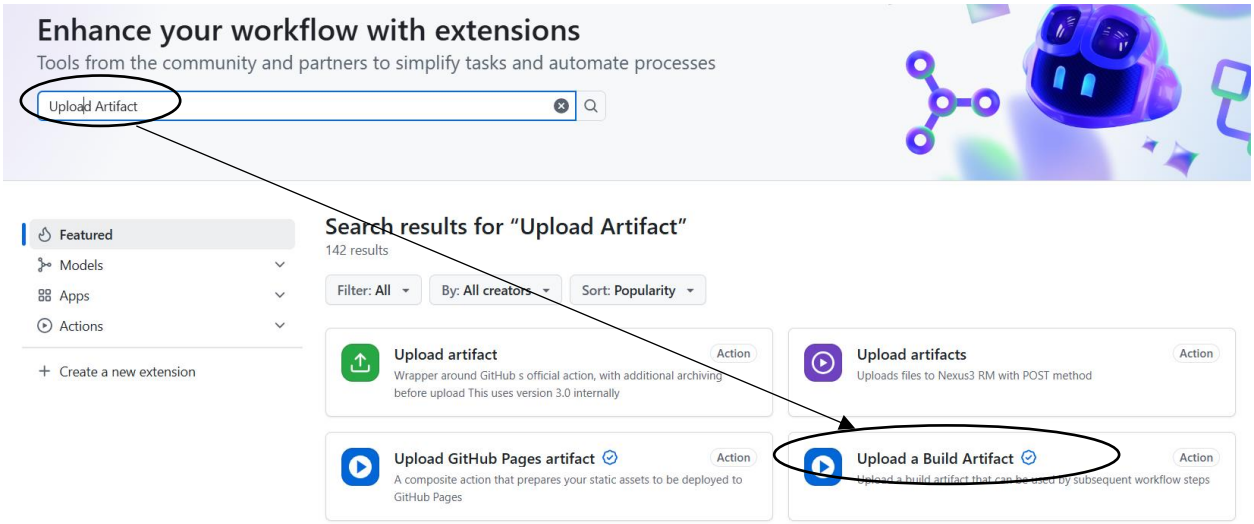
Real Example (CI/CD Flow)

- Job 1 – Build
  Install dependencies
  Run tests
  Build project

- Job 2 – Deploy
  Download artifact
  Deploy to server



This is to upload Artifact



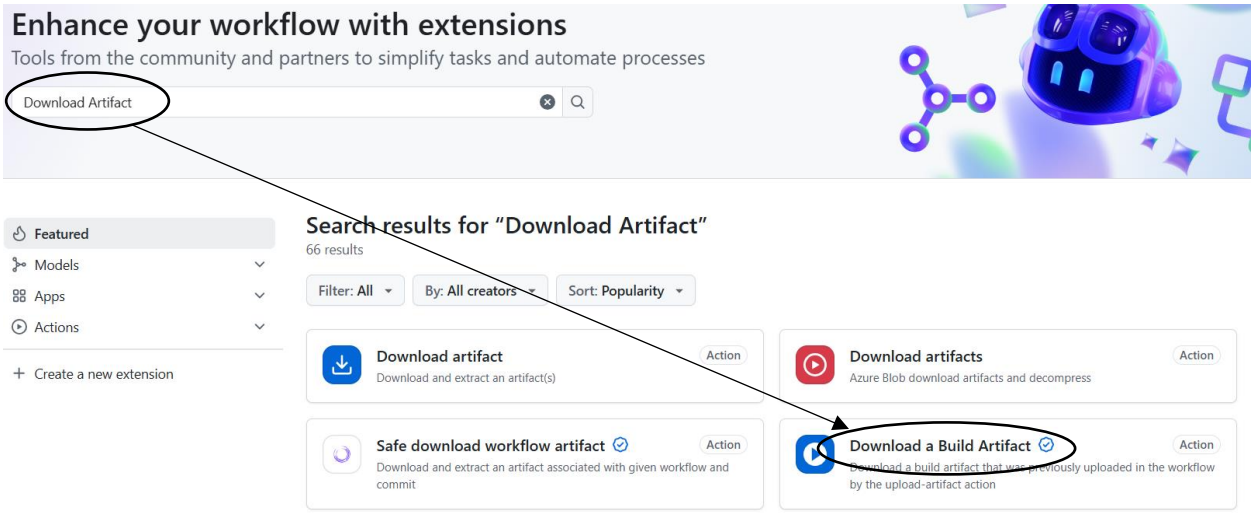
Usage

Inputs

Copy

```
- uses: actions/upload-artifact@v4
  with:
    # Name of the artifact to upload.
    # Optional. Default is 'artifact'
    name:
```

This Is to Download Artifact



Copy

Code:

```
name: Branches
# on: [push,workflow_dispatch]
on:
  push:
    branches:
      - main
      - feature/**
    paths-ignore:
      - README.md
      - '.github/workflows/**'
  workflow_dispatch:
 jobs:
    Run_Test:
       runs-on: ubuntu-latest
```

```
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Cache Dependences
        uses: actions/cache@v5
        with:
          path: ~/.npm
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Build:
    needs: Run_Test
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Cache Dependences
        uses: actions/cache@v5
        with:
          path: ~/.npm
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Install Dependences
        run: npm ci
      - name: Build Project
        run: npm run build
      - name: Upload Artifact
        uses: actions/upload-artifact@v4
        with:
          name: dist-files
          path: dist
  Deploy:
    needs: Build
    runs-on: ubuntu-latest
    steps:
      - name: Download Artifact
        uses: actions/download-artifact@v5
```

Here Is the code for Upload Artifact

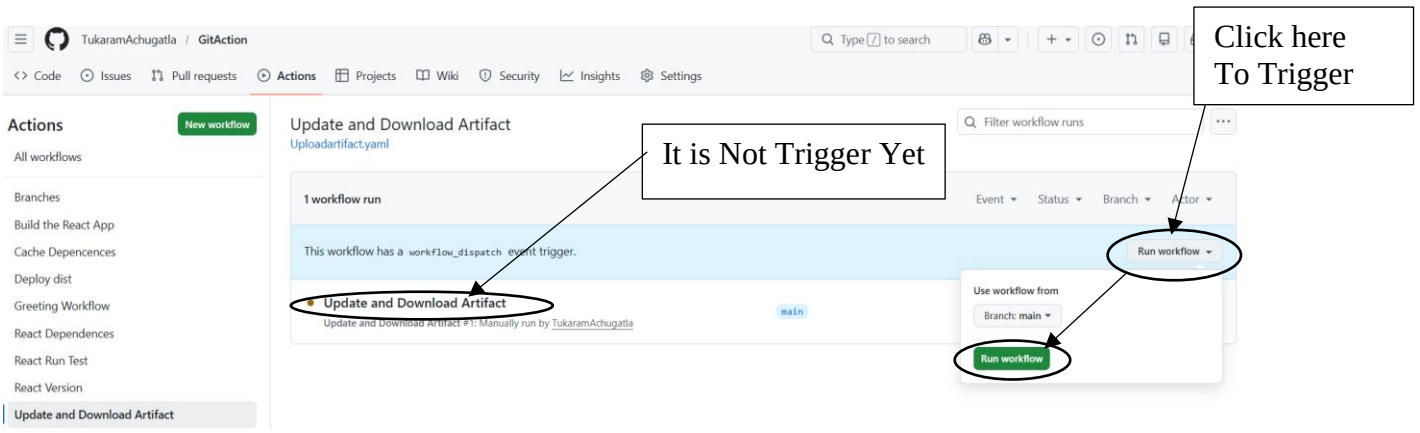
Here is the code For Download Artifact

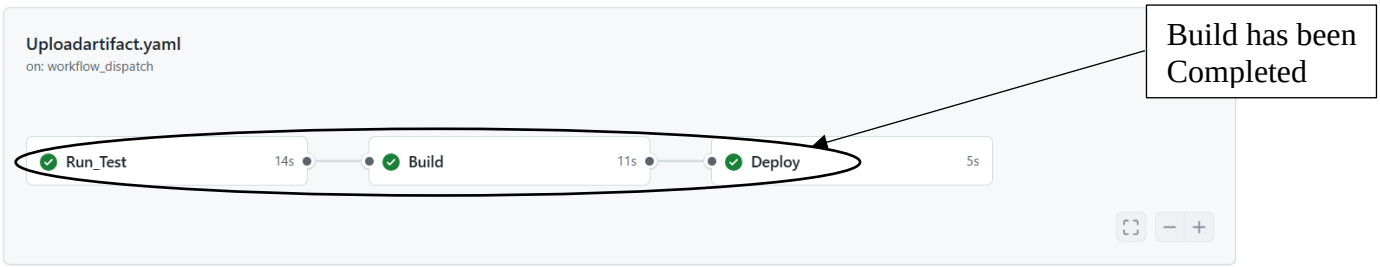
```
      with:
        name: dist-files
        path: dist
    - name: Get Code
      run: echo "Getting code..."
```

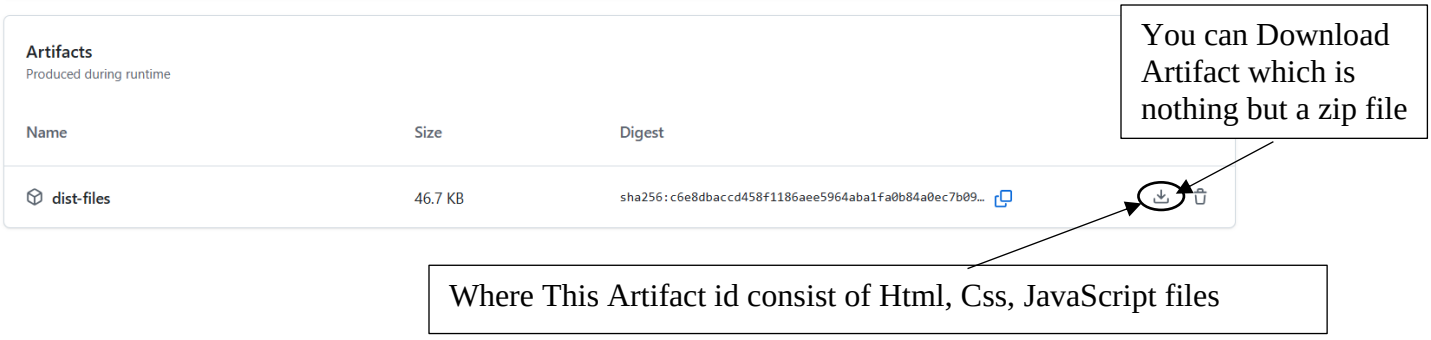
Note:-

Now, when we add code to GitHub and push it, the workflow will not trigger because of paths-ignore. We have specified that if changes are made only to those files, the workflow should not run.

Now open github→click on your repo →click on Action →





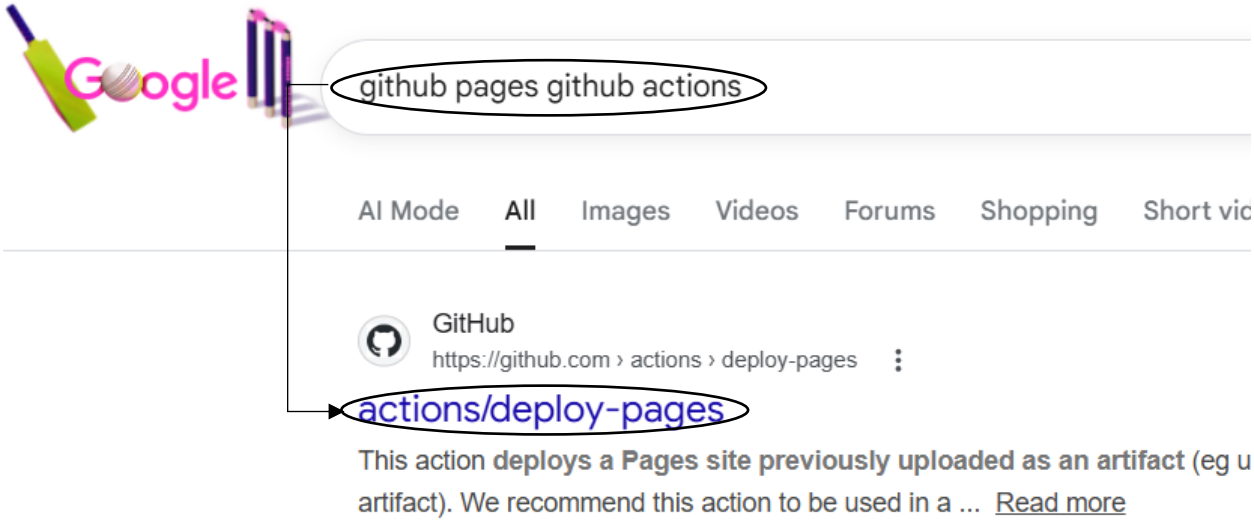


Deploy to GitHub pages using Github action

Now we are going to deploy a React application using GitHub Actions. We can deploy it through GitHub Pages, which allows us to host React or other web applications. Using GitHub Actions, we automate the deployment of the React application to GitHub Pages.

Create one file with DeployReactApp.yaml in .github/workflows

Search for github pages github action



Steps to Deploy the React Application Using GitHub Pages

1. **Scroll down to the "Usage" section** in the GitHub Pages documentation.
2. **Copy the code from the "Deploy" section** and paste it into your DeployReactApp.yaml file.
3. In the build job, **modify the upload artifact step** as shown below

   - name: Upload Artifact
   uses: actions/upload-pages-artifact@v3
   with:
   name: github-pages
   path: dist
4. Next, open the package.json file and add the following homepage URL
   "homepage: https://TukaramAchugatla.github.io/GitAction"

This is my Github user Name

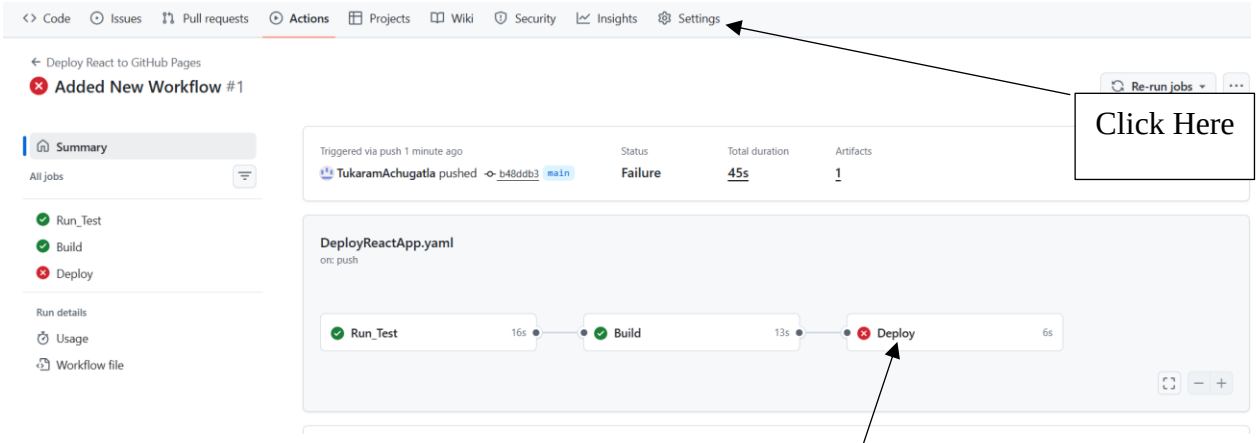
This is my Github Repository Name

5. Next, open the vite.config.js file and add your repository name in the base property as shown below:
base: '/GitAction/',

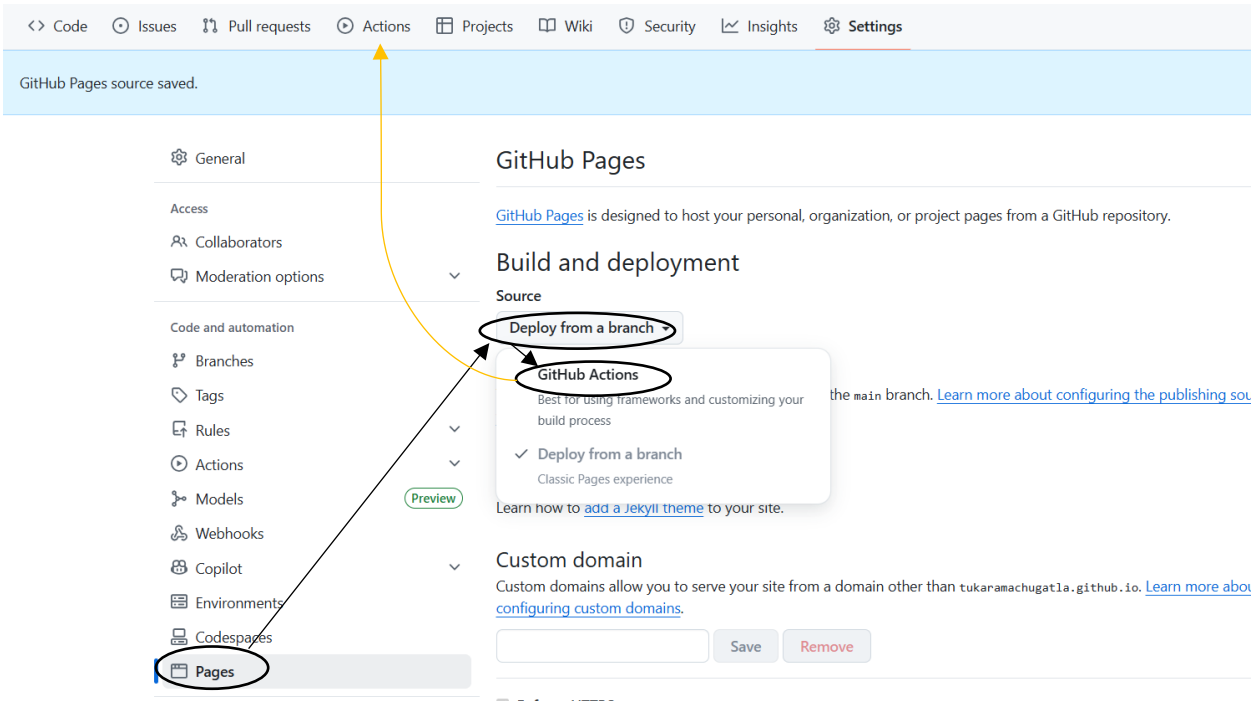
Code :-

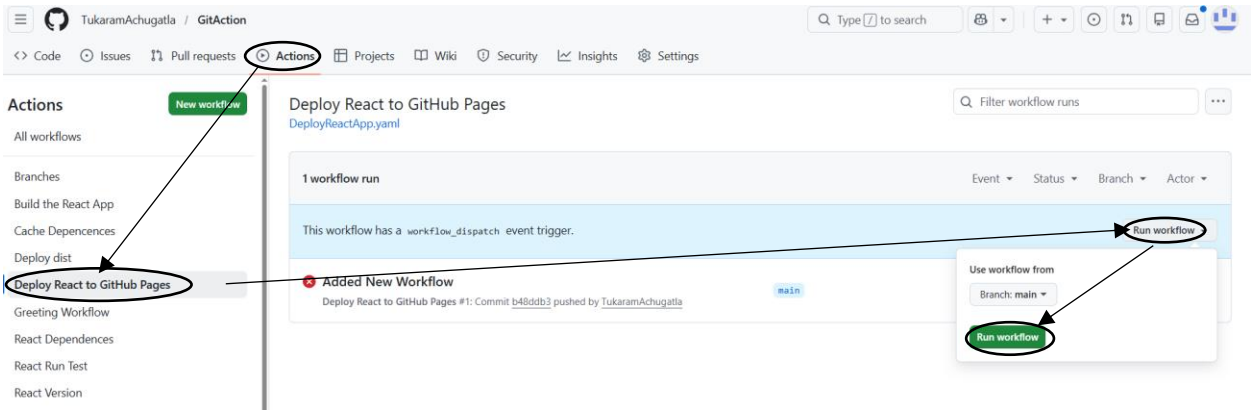
```
name: Deploy React to GitHub Pages
on:
  push:
    branches:
      - main
      - feature/**
    paths-ignore:
      - README.md
      - '.github/workflows/**'
  workflow_dispatch:
jobs:
    Run_Test:
        runs-on: ubuntu-latest
        steps:
            - name: Get Code
              uses: actions/checkout@v6
            - name: Install Node
              uses: actions/setup-node@v6
              with:
                node-version: '20'
            - name: Cache Dependences
              uses: actions/cache@v5
              with:
                path: ~/.npm
                key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
            - name: Install Dependences
              run: npm ci
            - name: Run Test
              run: npm test
```

```
Build:
  needs: Run_Test
  runs-on: ubuntu-latest
  steps:
    - name: Get Code
      uses: actions/checkout@v6
    - name: Install Node
      uses: actions/setup-node@v6
      with:
        node-version: '20'
    - name: Cache Dependences
      uses: actions/cache@v5
      with:
        path: ~/.npm
        key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
    - name: Install Dependences
      run: npm ci
    - name: Build Project
      run: npm run build
    - name: Upload Artifact
      uses: actions/upload-pages-artifact@v3
      with:
        name: github-pages
        path: dist
Deploy:
  needs: Build
  runs-on: ubuntu-latest


  # Grant GITHUB_TOKEN the permissions required to make a Pages deployment
  permissions:
    pages: write      # to deploy to Pages
    id-token: write   # to verify the deployment originates from an appropriate source

  # Deploy to the github-pages environment
  environment:
    name: github-pages
    url: ${{ steps.deployment.outputs.page_url }}

  steps:
  - name: Deploy to GitHub Pages
    id: deployment
    uses: actions/deploy-pages@v4
    with:
      tocken: ${{ secrets.GITHUB_TOKEN }}
```

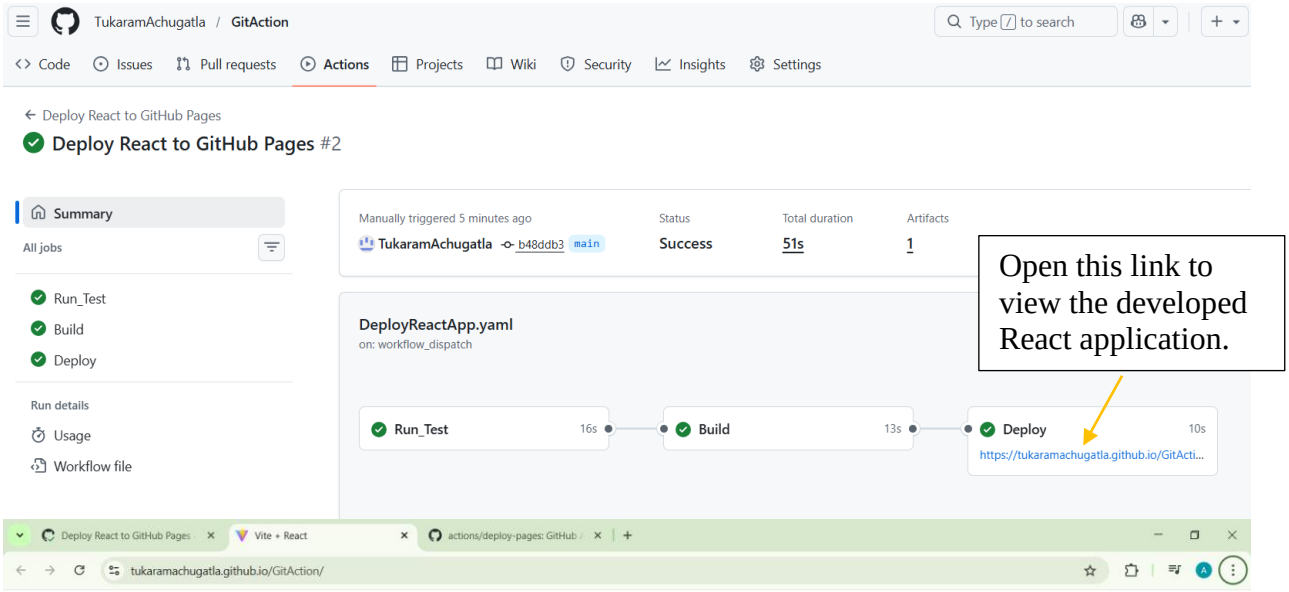


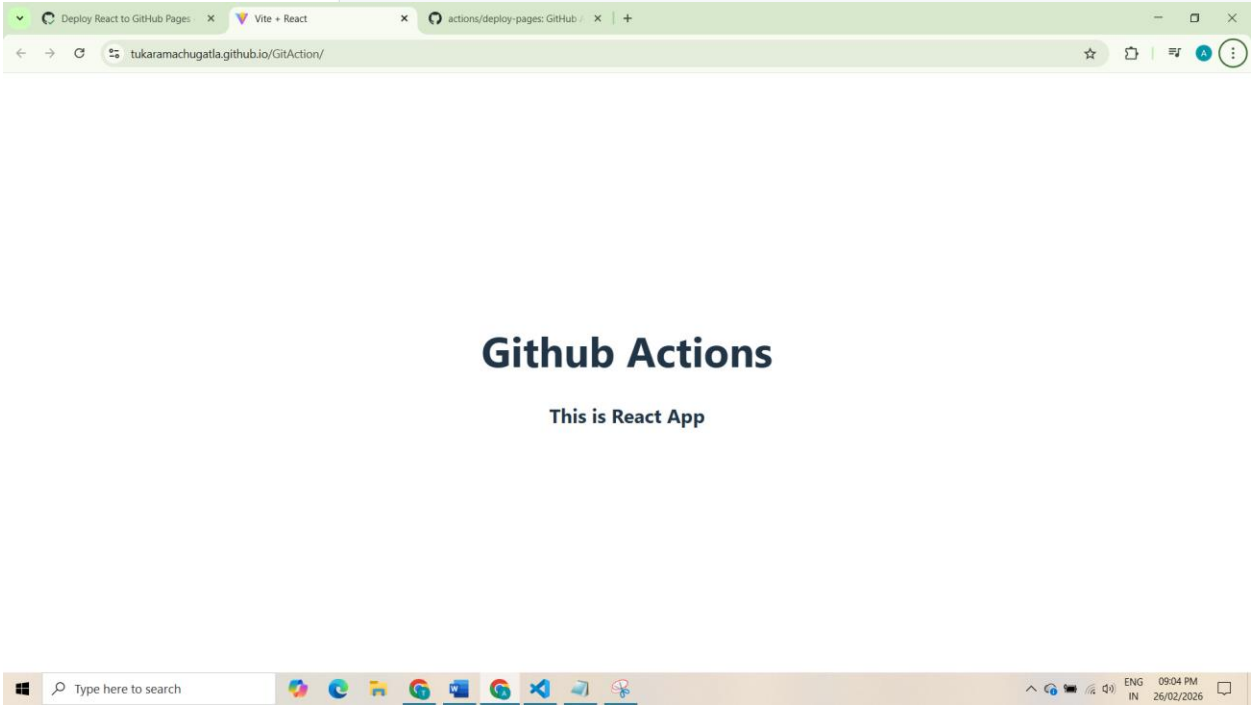
If the workflow fails as shown above, go to the Settings tab




TukaramAchugatla / GitAction
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
Branches
Build the React App
Cache Depencences
Deploy dist
Deploy React to GitHub Pages
Greeting Workflow
React Dependences
React Run Test
React Version
Deploy React to GitHub Pages
DeployReactApp.yaml
Filter workflow runs
1 workflow run
Event
Status
Branch
Actor
This workflow has a workflow_dispatch event trigger.
Run workflow
Added New Workflow
Deploy React to GitHub Pages #1: Commit b48ddb3 pushed by TukaramAchugatla
main
Use workflow from
Branch: main
Run workflow

Now its Done


TukaramAchugatla / GitAction
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Deploy React to GitHub Pages
Deploy React to GitHub Pages #2
Summary
All jobs
Run_Test
Build
Deploy
Run details
Usage
Workflow file
Manually triggered 5 minutes ago
TukaramAchugatla b48ddb3 main
Status
Success
Total duration
51s
Artifacts
1
DeployReactApp.yaml
on: workflow_dispatch
Run_Test 16s
Build 13s
Deploy 10s
https://tukaramachugatla.github.io/GitActi...
Open this link to view the developed React application.
Deploy React to GitHub Pages
Vite + React
actions/deploy-pages: GitHub
tukaramachugatla.github.io/GitAction/


Github Actions
This is React App
Type here to search
ENG
IN
09:04 PM
26/02/2026

Integrate Sonar Cloud with Git action

Case Study:-

If any developer pushes their code to a feature branch and raises a Pull Request (PR), we first check the code quality using SonarCloud integration before merging.

If the code quality is not good, I will not approve the PR and will inform the developer about the issues.

Once the developer fixes the issues and meets the required quality standards, I will review and approve the code.

Steps to Integrate SonarCloud with GitHub

SonarCloud Account Setup

1. Search for **SonarCloud** on Google.
2. Log in using your **GitHub account**.
3. Click on **Authorize SonarQubeCloud**.
4. Select the **Free organization plan** and create the organization.
5. Click on **"Get Started with GitHub."**
6. Choose **All repositories**, then click **Install & Authorize**.
7. Enter your GitHub password and click **Confirm**.
8. Scroll down, select the **Free organization plan**, and click **Create**.
9. Click on **Analyze New Project**.
10. Select the project you want to integrate (for example, **GitAction**) and click **Set Up**.
11. Click on **Previous Version**, then click **Create Project**.

Configure SonarCloud in GitHub

12. Open the **SonarQube Cloud Dashboard**.
13. Click on **Administration → Analysis Method**.
14. Enable **Automatic Analysis**.
15. Click on your **repository name**.
16. Copy the **Name** and **Value** fields and save them in Notepad.

Add SonarCloud Secrets in GitHub

17. Go to your GitHub repository → **Settings**.
18. Click on **Secrets and Variables → Actions**.
19. Click on **New repository secret**.
20. Paste the copied **Name** and **Value** fields.
21. Click on **Add Secret**.

Add SonarCloud Workflow File

22. Go back to the SonarCloud dashboard and scroll down to **Other CI**.
23. Copy the provided GitHub Actions YAML code.
24. In VS Code, create a new file named .github/workflows/sonarIssues.yaml
25. Paste the copied YAML code into this file.

Add sonar-project.properties File

26. In the SonarCloud dashboard, scroll down and copy the **sonar-project.properties** content.
27. Create a new file in your project root (outside .github/workflows) named sonar-project.properties

28. Paste the copied content into this file.

Run the Workflow

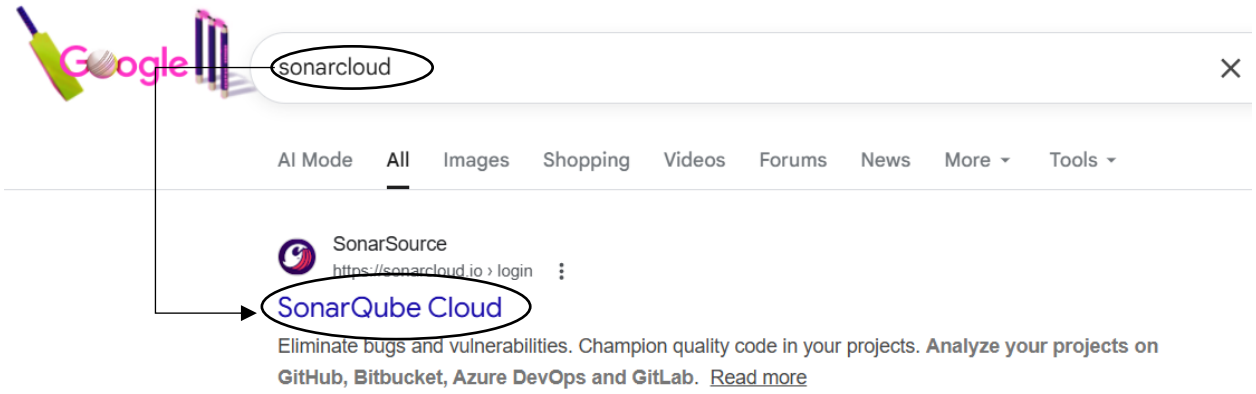
29. Push the code to GitHub.
30. Go to the **Actions tab** and verify that the SonarCloud workflow runs successfully

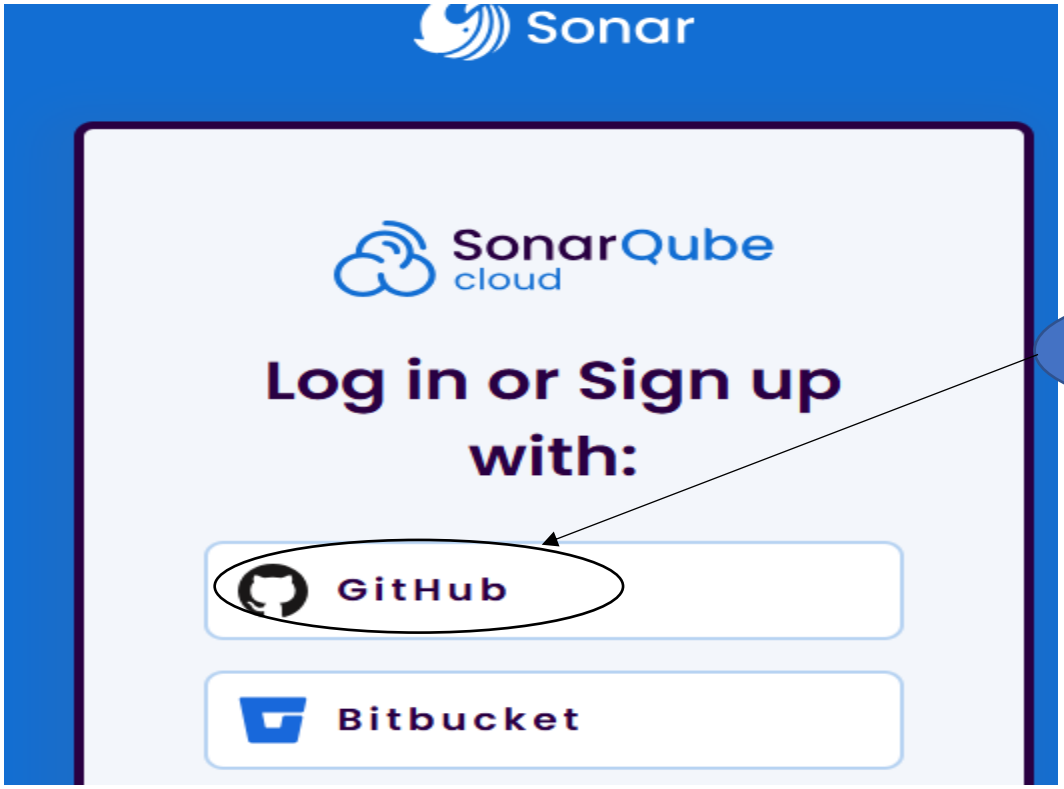
Create New Branch and Raise PR

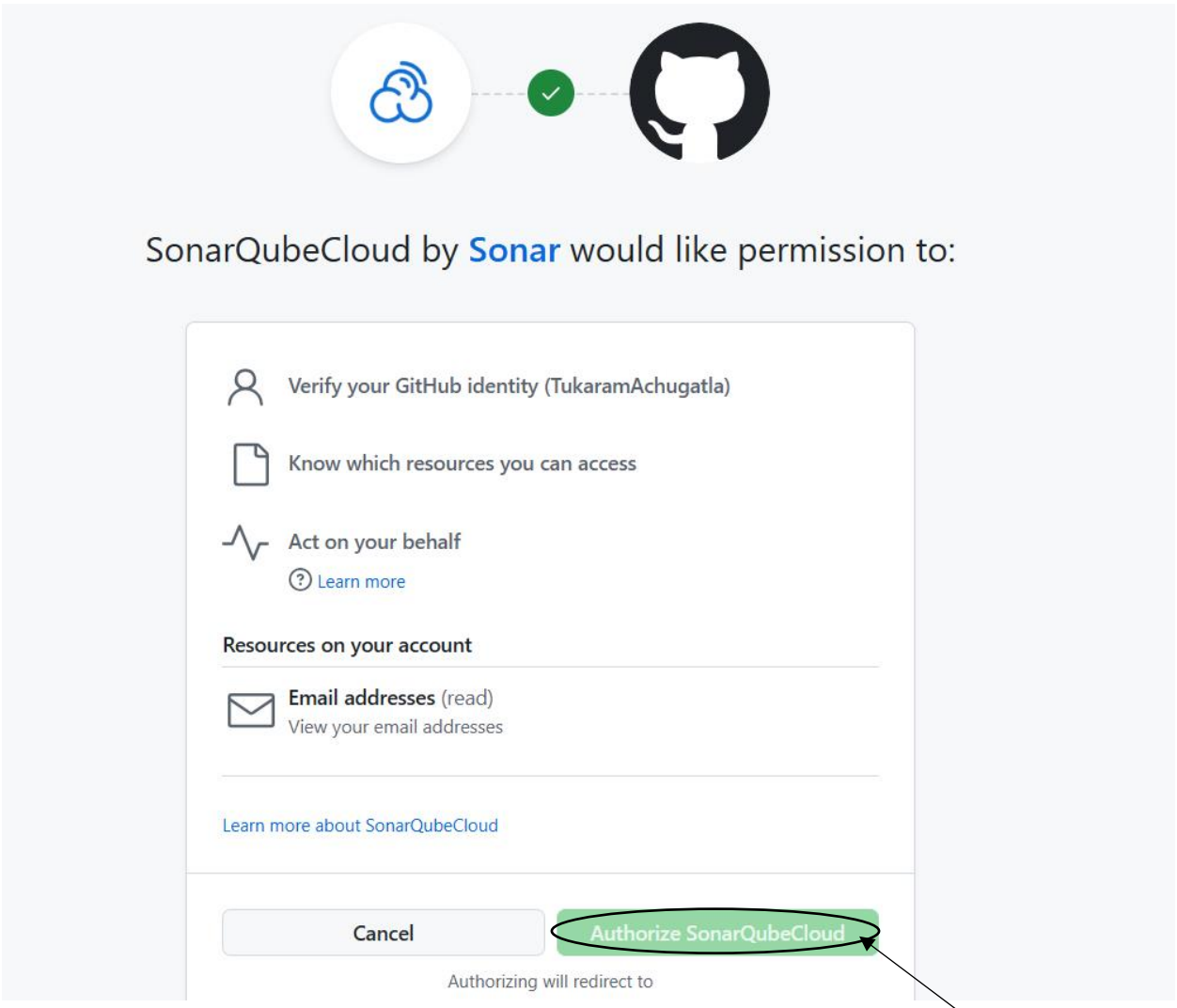
31. Create a new branch named newh5tag
32. Add the following tag in App.jsx <h5>with SonarCloud</h5>
33. Push the branch to GitHub.
34. Raise a **Pull Request (PR)** from newh5tag to main.
35. Wait for the SonarQubeCloud check to complete.
36. If the Quality Gate passes, click **Merge**.

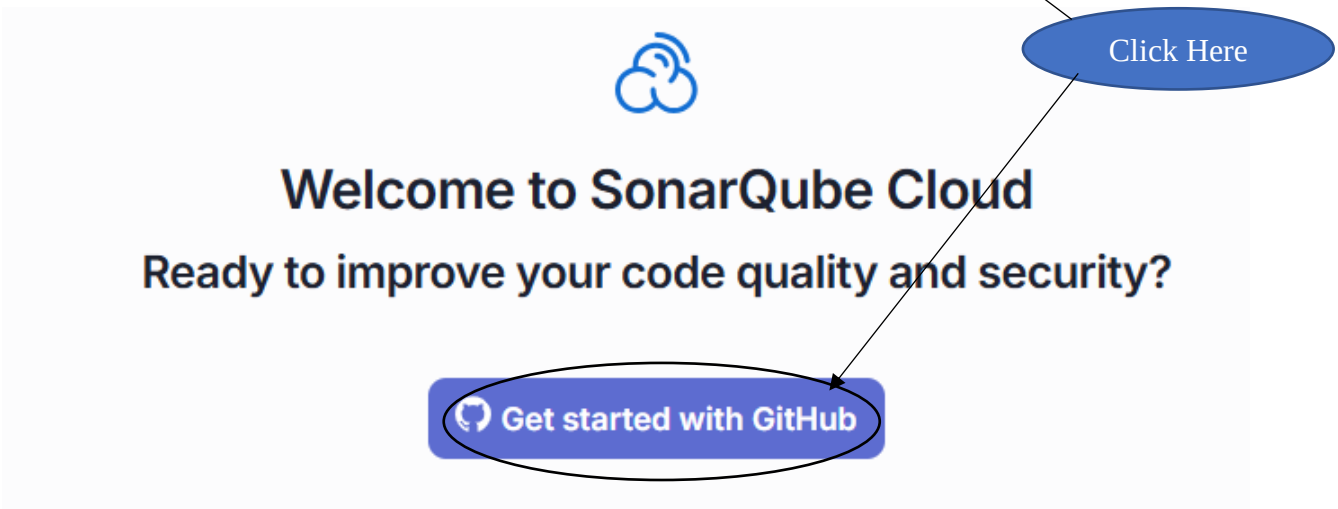
Final Result

37. After merging, SonarQubeCloud should pass successfully.
38. Finally, open the **SonarQube Cloud dashboard** to view the complete analysis report.

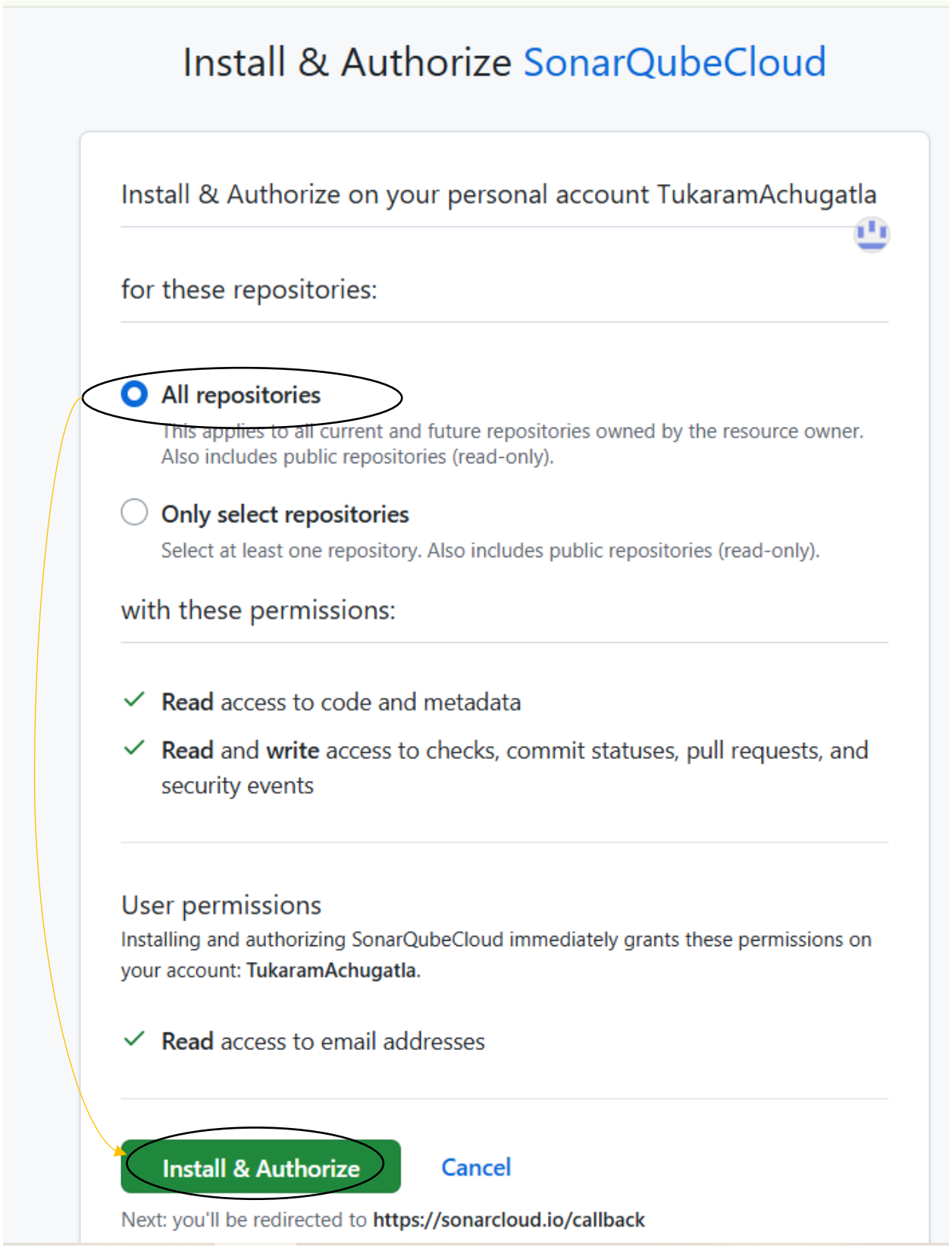

Google
sonarcloud
AI Mode
All
Images
Shopping
Videos
Forums
News
More
Tools
SonarSource
https://sonarcloud.io › login
SonarQube Cloud
Eliminate bugs and vulnerabilities. Champion quality code in your projects. Analyze your projects on GitHub, Bitbucket, Azure DevOps and GitLab. Read more

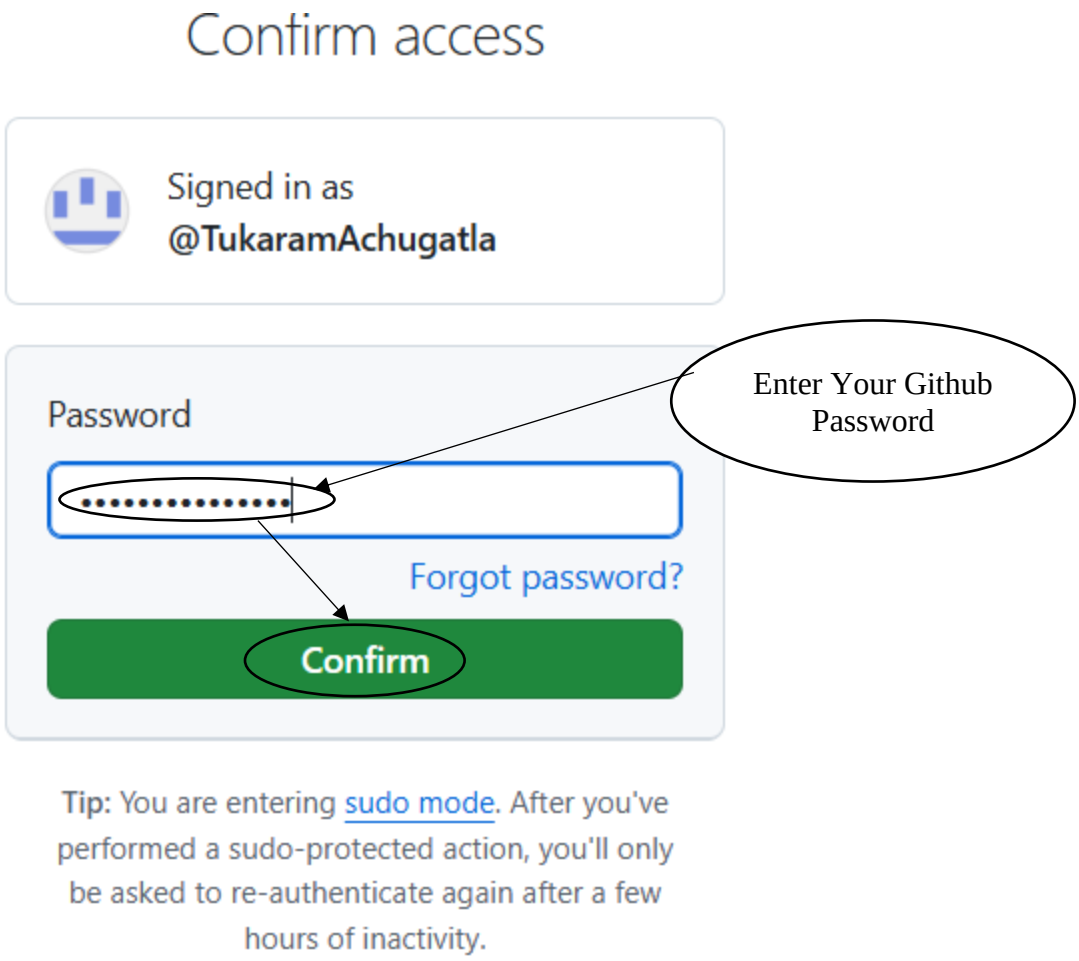

Sonar
SonarQube cloud
Log in or Sign up with:
GitHub
Bitbucket
Click Here

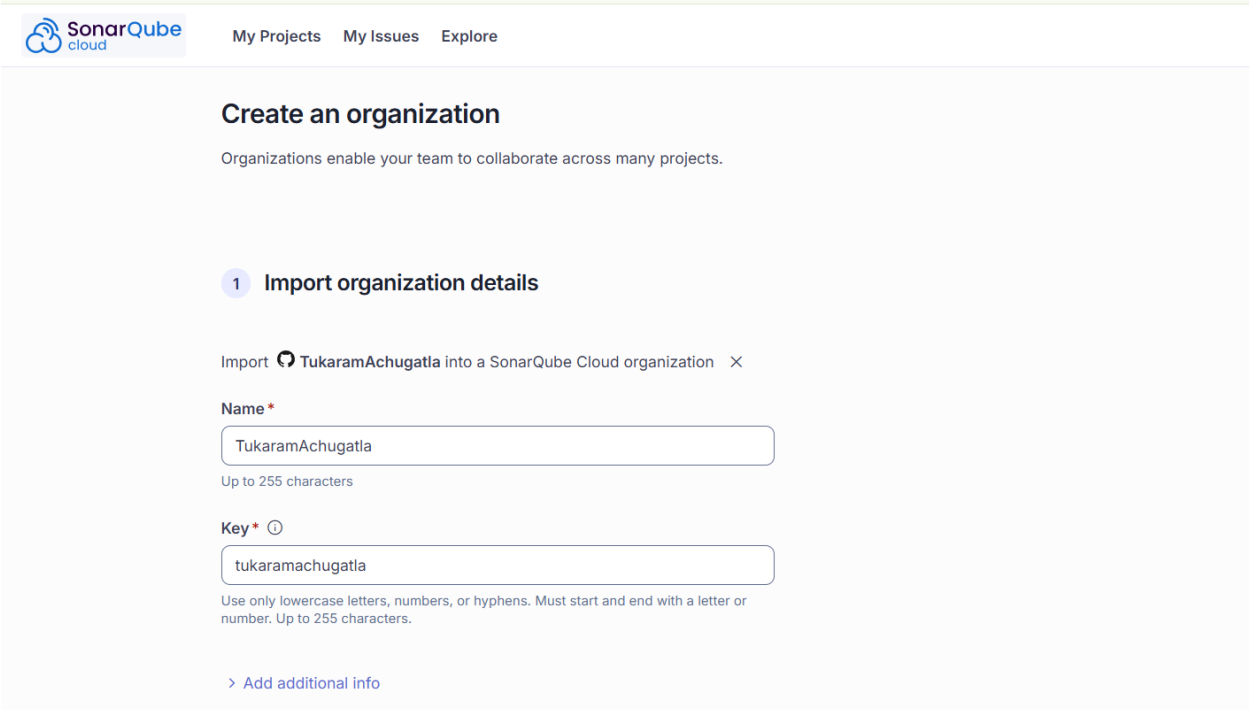

SonarQubeCloud by Sonar would like permission to:
Verify your GitHub identity (TukaramAchugatla)
Know which resources you can access
Act on your behalf
Learn more
Resources on your account
Email addresses (read)
View your email addresses
Learn more about SonarQubeCloud
Cancel
Authorize SonarQubeCloud
Authorizing will redirect to

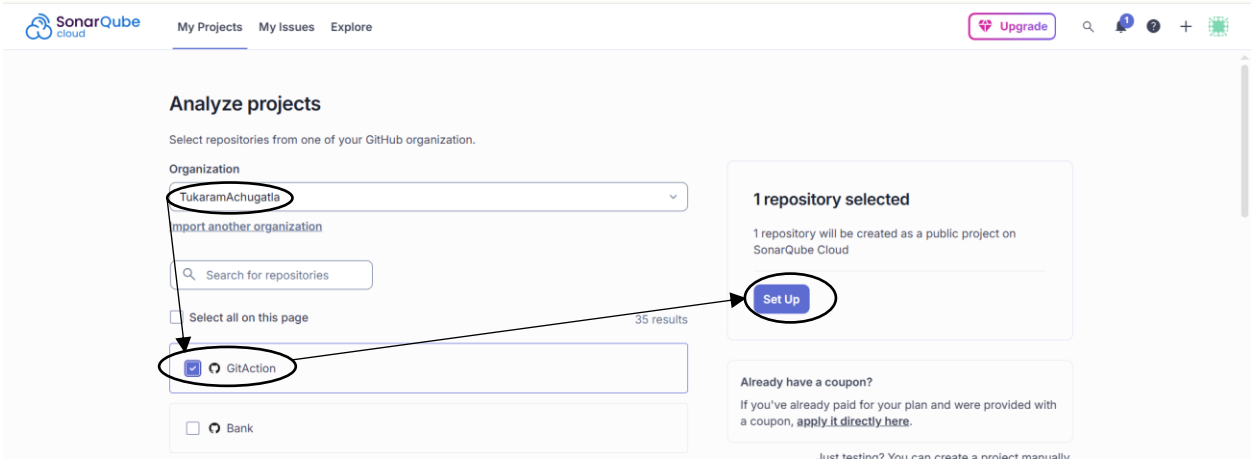

Welcome to SonarQube Cloud
Ready to improve your code quality and security?
Get started with GitHub

Click Here


Install & Authorize SonarQubeCloud
Install & Authorize on your personal account TukaramAchugatla
for these repositories:
All repositories
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).
Only select repositories
Select at least one repository. Also includes public repositories (read-only).
with these permissions:
Read access to code and metadata
Read and write access to checks, commit statuses, pull requests, and security events
User permissions
Installing and authorizing SonarQubeCloud immediately grants these permissions on your account: TukaramAchugatla.
Read access to email addresses
Install & Authorize
Cancel
Next: you'll be redirected to https://sonarcloud.io/callback


Confirm access
Signed in as
@TukaramAchugatla
Password
Enter Your Github Password
Forgot password?
Confirm
Tip: You are entering sudo mode. After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.


SonarQube cloud
My Projects
My Issues
Explore
Create an organization
Organizations enable your team to collaborate across many projects.
1 Import organization details
Import TukaramAchugatla into a SonarQube Cloud organization
Name *
TukaramAchugatla
Up to 255 characters
Key *
tukaramachugatla
Use only lowercase letters, numbers, or hyphens. Must start and end with a letter or number. Up to 255 characters.
Add additional info



Set up new code for project

The new code definition sets the criteria for what's considered new code, allowing you to focus on the most recent changes in your project.

This setting allows project administrators to customize the new code definition for their project.
Learn more in documentation

Your organization's new code definition is not set

Set a new code definition for your organisation to use it by default for all new projects

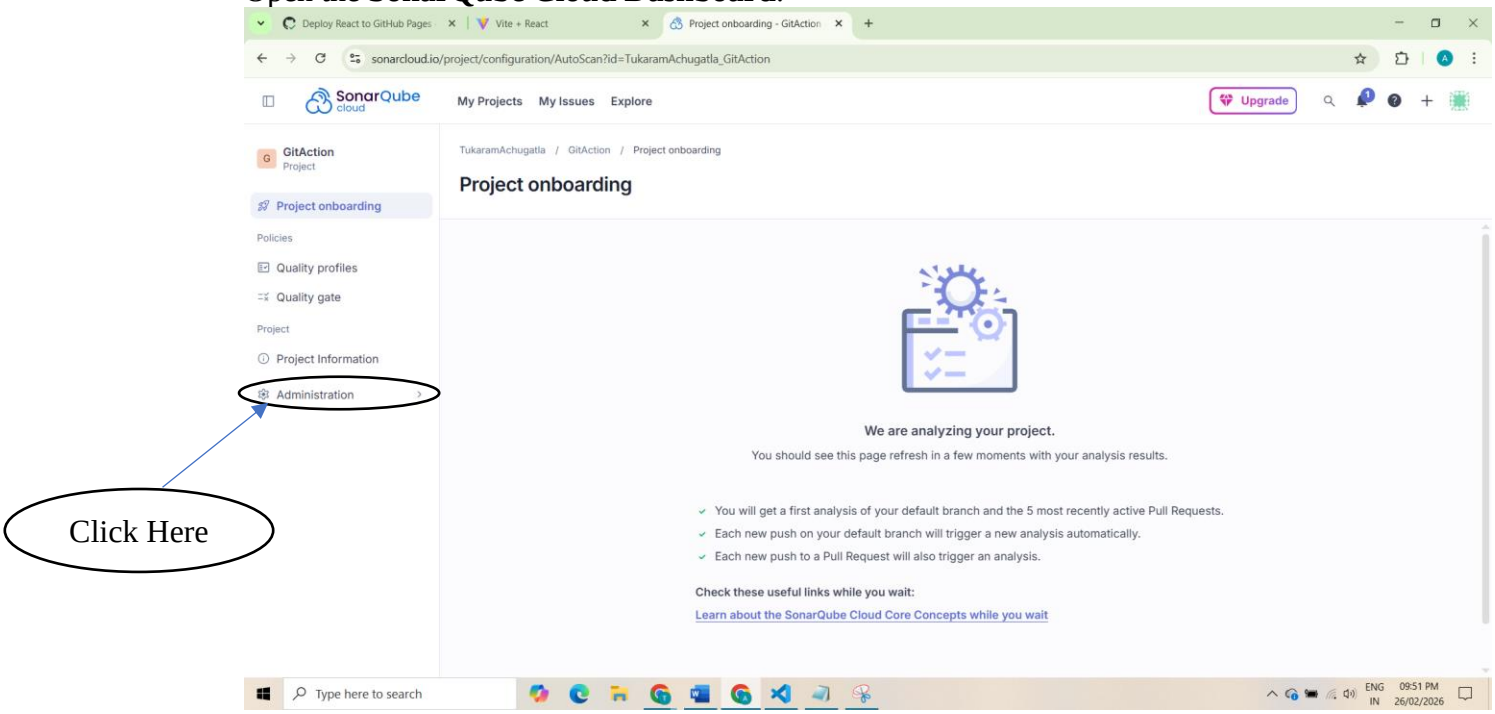
TukaramAchugatla - Administration - New Code

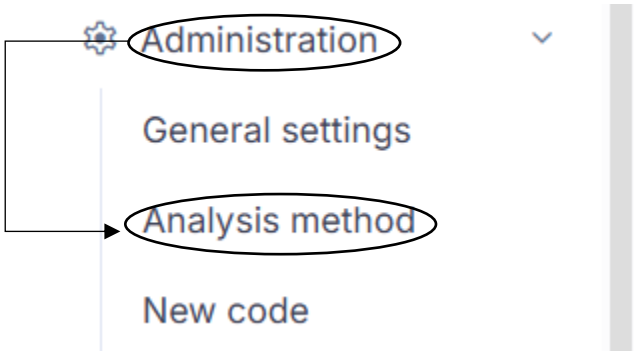
New code definition for this project:

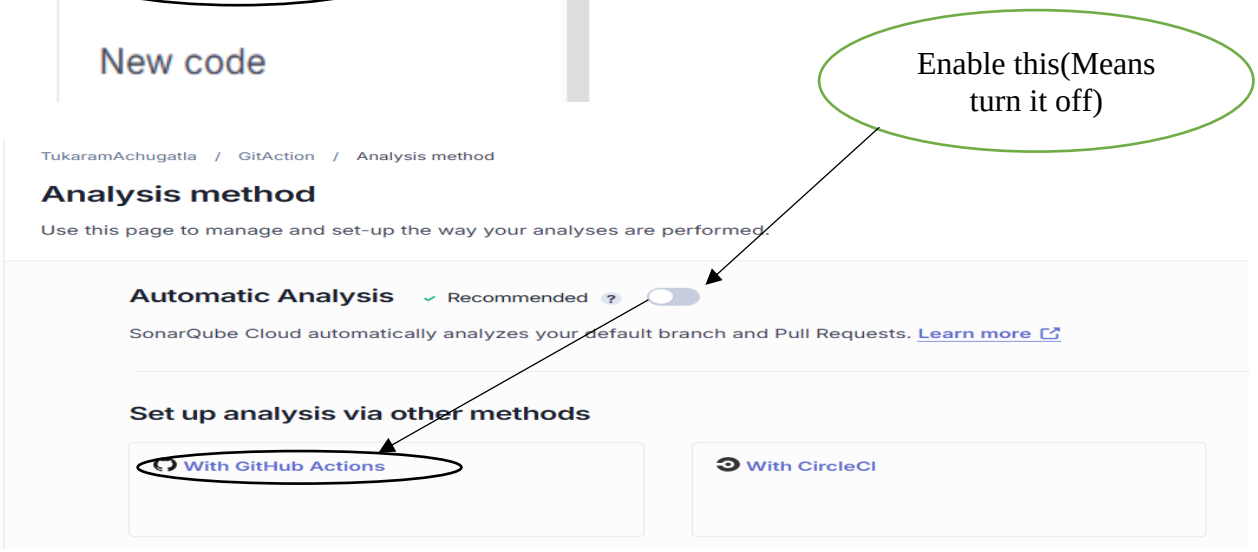
- (●) **Previous version**
  Any code that has changed since the previous version is considered new code. Recommended for **projects following regular versions or releases.**
- ( ) **Number of days**
  Any code that has changed in the last x days is considered new code. If no action is taken on a new issue after x days, this issue will become part of the overall code. Recommended for **projects following continuous delivery.**

Back | Create project

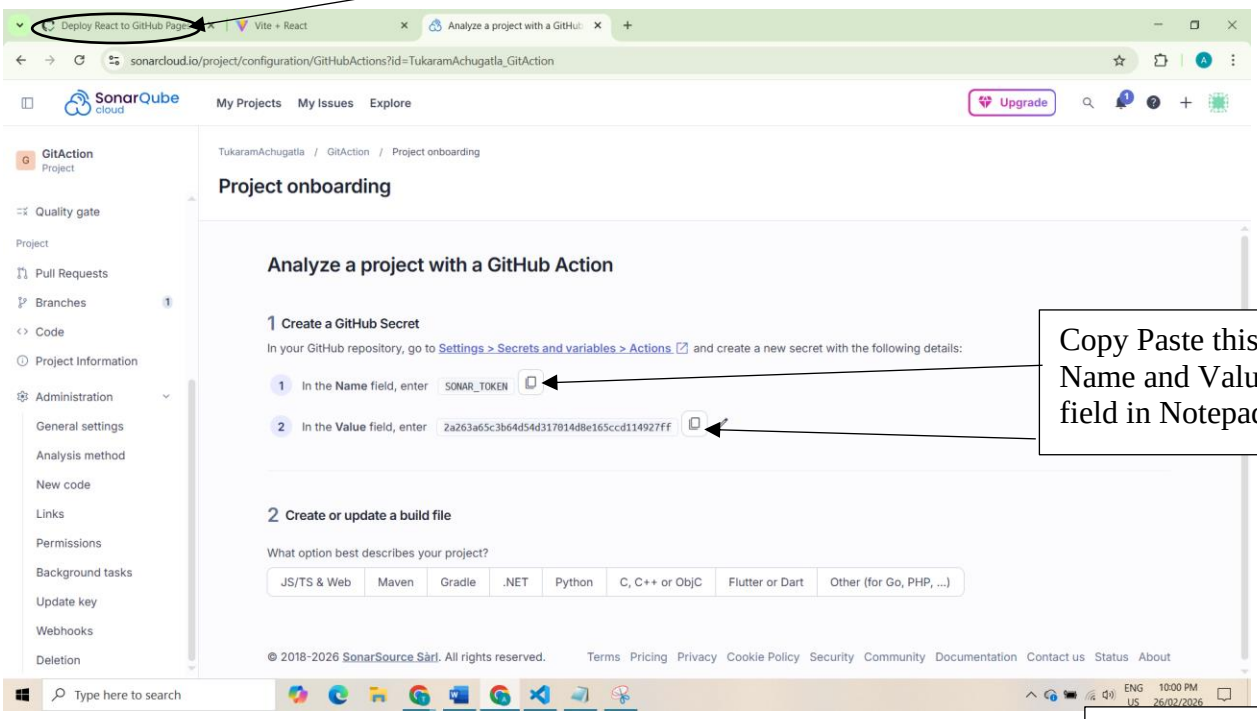
Open the **SonarQube Cloud Dashboard**.





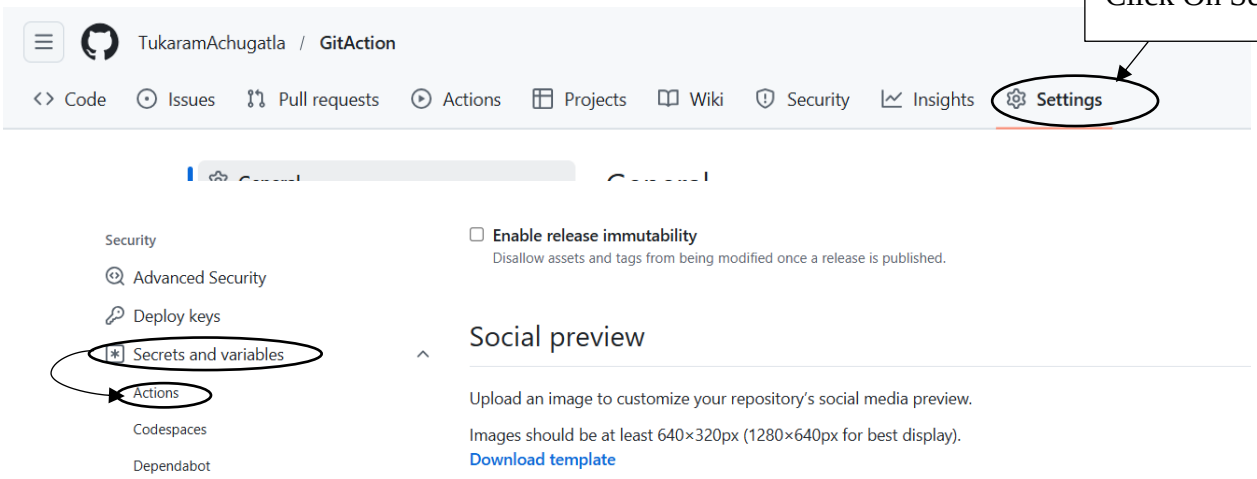


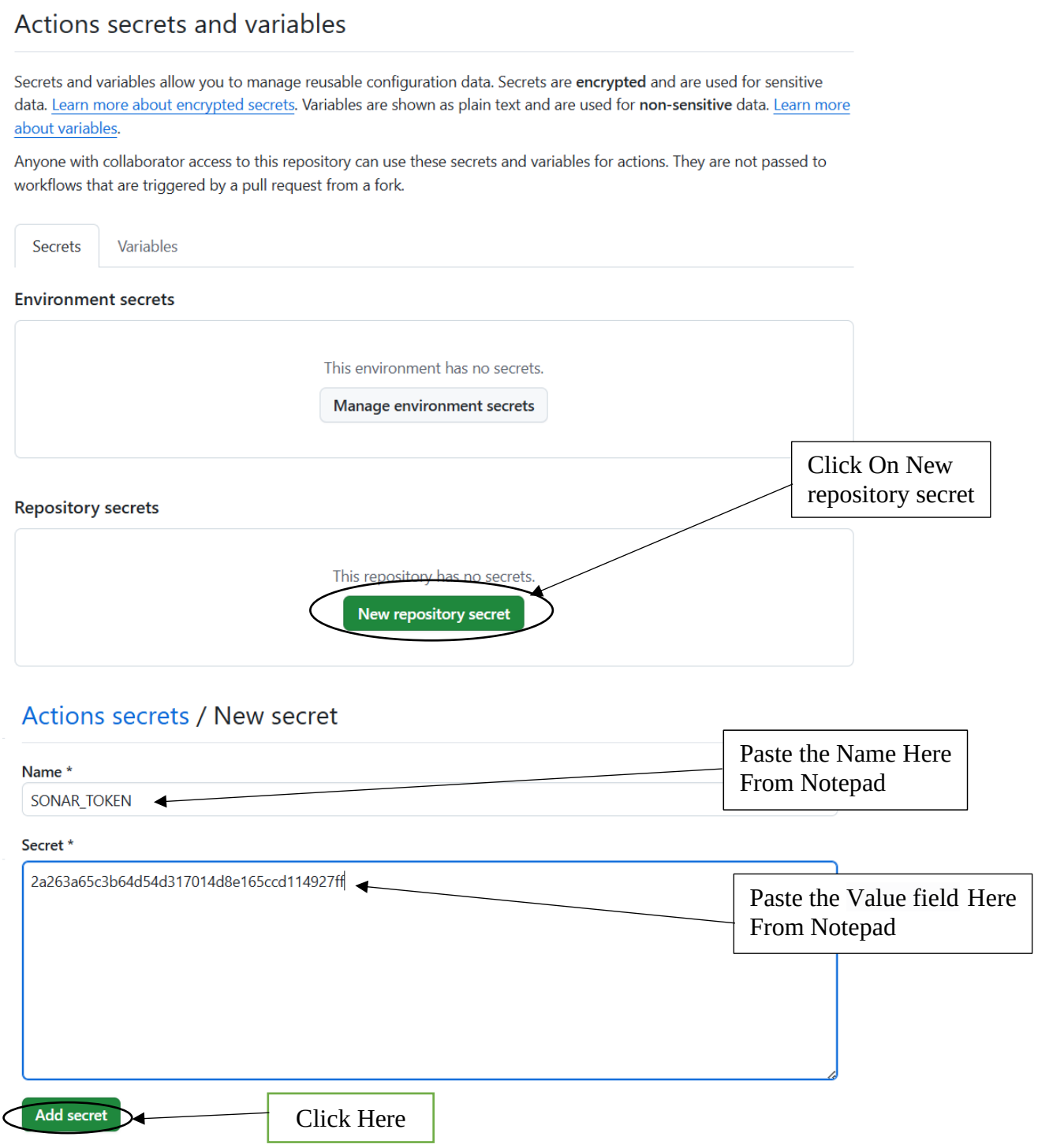
Now go to your Github repositories



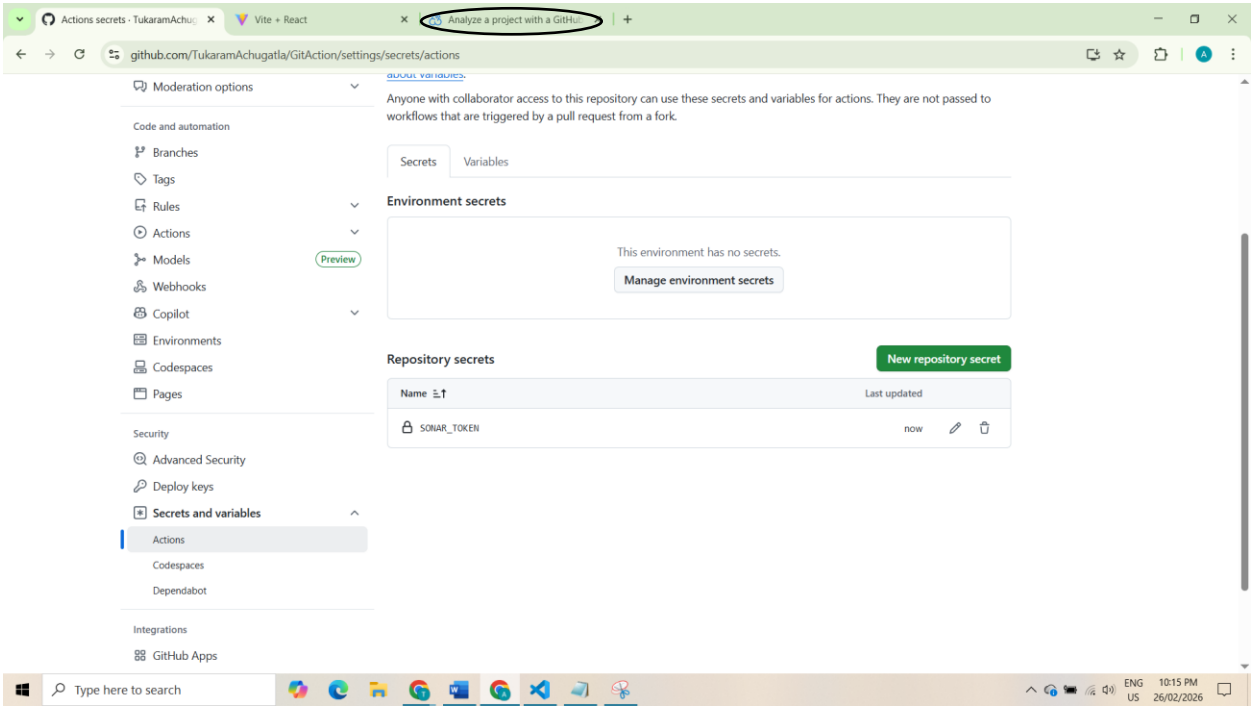
Copy Paste this Name and Value field in Notepad

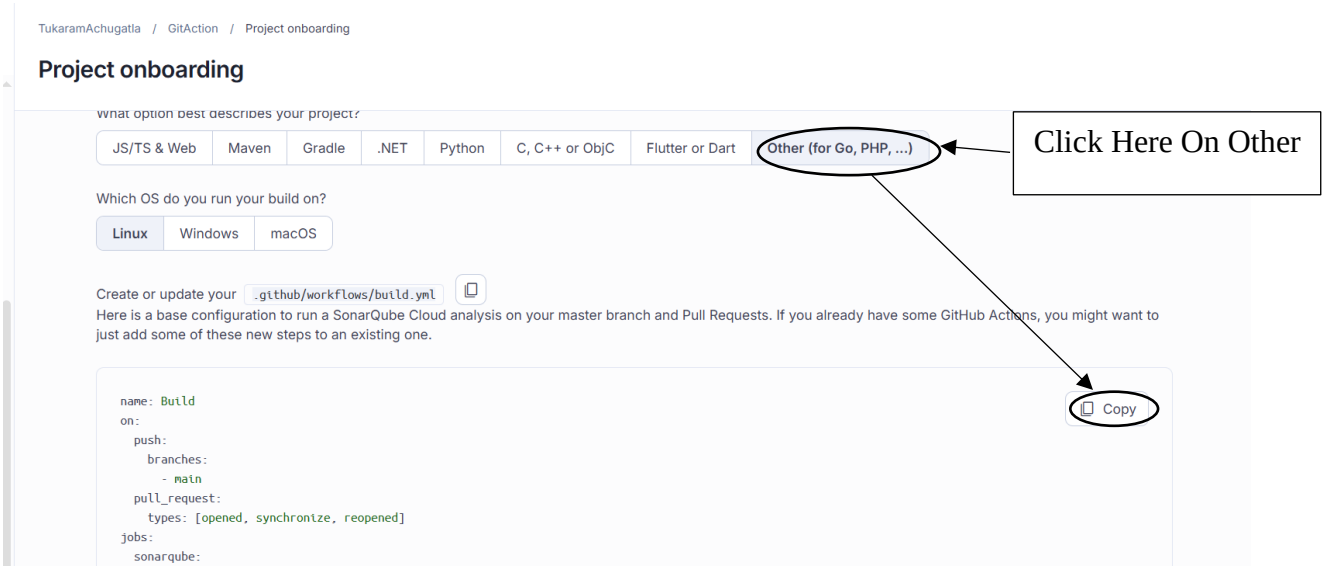
Click On Setting




Actions secrets and variables
Secrets and variables allow you to manage reusable configuration data. Secrets are encrypted and are used for sensitive data. Learn more about encrypted secrets. Variables are shown as plain text and are used for non-sensitive data. Learn more about variables.
Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.
Secrets
Variables
Environment secrets
This environment has no secrets.
Manage environment secrets
Repository secrets
This repository has no secrets.
New repository secret
Click On New repository secret
Actions secrets / New secret
Name *
SONAR_TOKEN
Paste the Name Here From Notepad
Secret *
2a263a65c3b64d54d317014d8e165ccd114927ff
Paste the Value field Here From Notepad
Add secret
Click Here

- Go back to the SonarCloud dashboard and scroll down to **Other CI**.

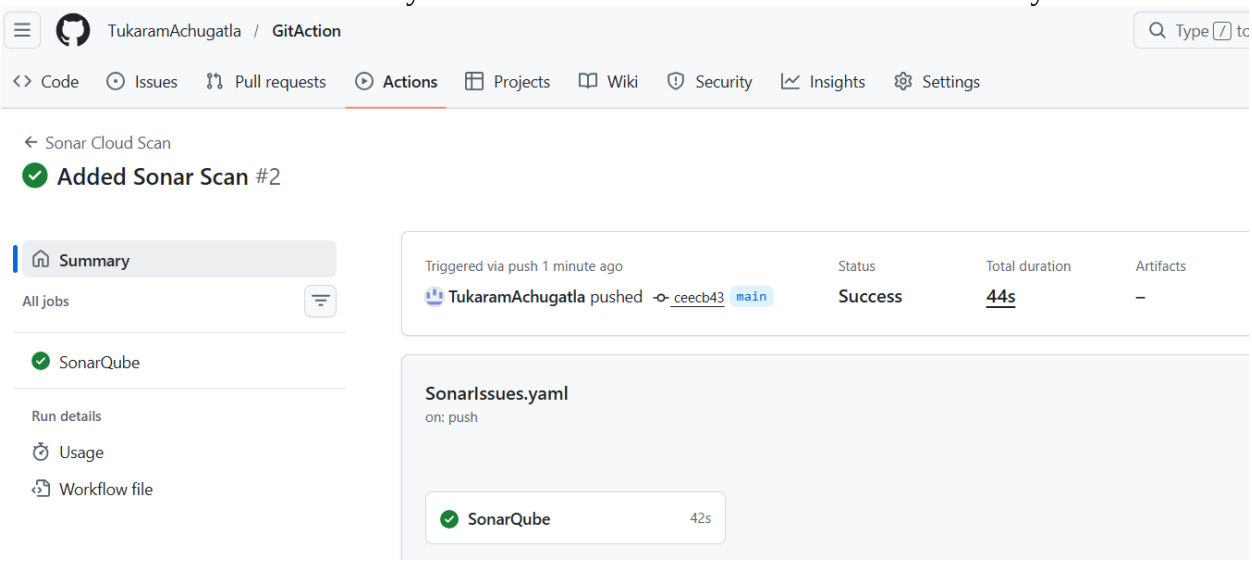




- Copy the provided GitHub Actions YAML code.
- In VS Code, create a new file named .github/workflows/sonarIssues.yaml
- Paste the copied YAML code into this file.
- In the SonarCloud dashboard, scroll down and copy the **sonar-project.properties** content.
- Create a new file in your project root (outside .github/workflows) named sonar-project.properties

- Paste the copied content into this file.

- Push the code to GitHub
- Go to the **Actions tab** and verify that the SonarCloud workflow runs successfully.



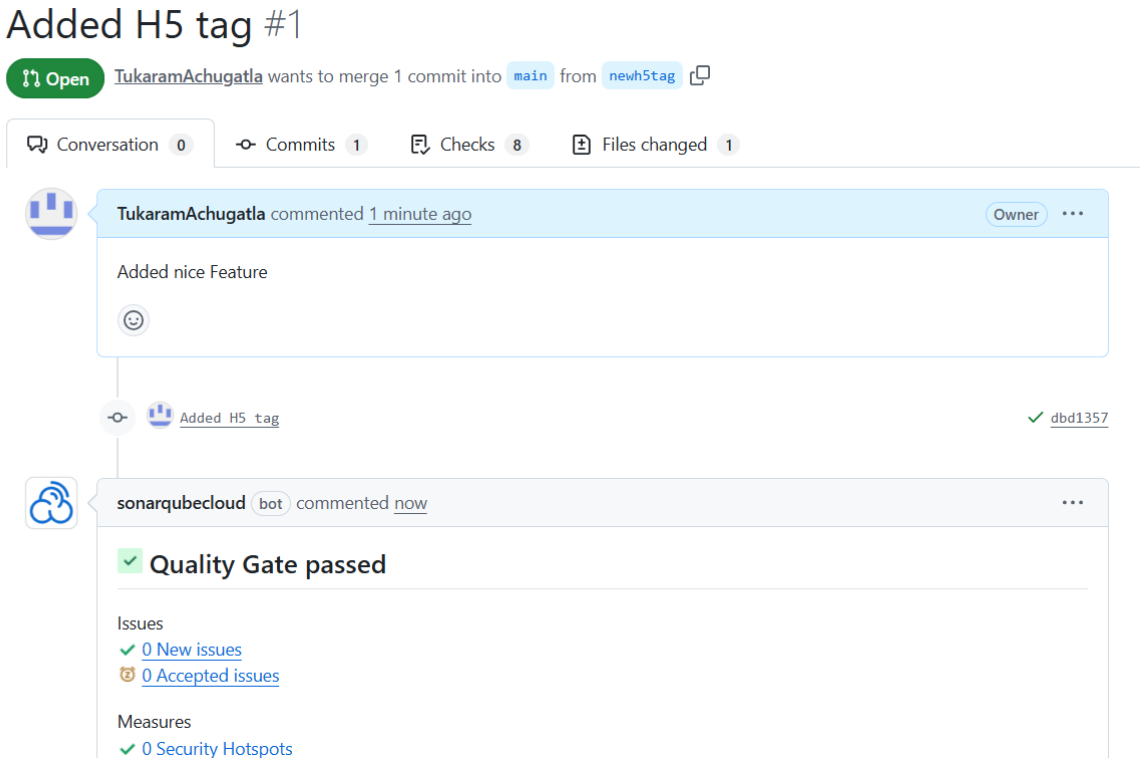
Create a new branch named “newh5tag”

```
To https://github.com/TukaramAchugatla/GitAction.git
   e849833..ceecb43  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\HP\Desktop\Kubernets\Git Action\cicd-github-actions-project> git checkout -b "newh5tag"
Switched to a new branch 'newh5tag'
PS C:\Users\HP\Desktop\Kubernets\Git Action\cicd-github-actions-project>
```

Add the following tag in App.jsx :<h5>with SonarCloud</h5>

```
<>
  <h1>Github Actions</h1>
  <h2>This is React App</h2>
  <h5>with SonarCloud</h5>
</>
```

- Push the branch to GitHub.
- Raise a **Pull Request (PR)** from `newh5tag` to `main`.
- Wait for the SonarQubeCloud check to complete.
- If the Quality Gate passes, click **Merge**.
- After merging, SonarQubeCloud should pass successfully.



- Finally, open the **SonarQube Cloud dashboard** to view the complete analysis report